

GoSSamer: Lightweight and Linear-Communication Asynchronous (Dynamic Proactive) Secret Sharing and the Applications

Xinxin Xing*, Yizhong Liu*, Boyang Liao*, Jianwei Liu*, Bin Hu*, Xun Lin*, Yuan Lu[†], Tianwei Zhang[‡]

*Beihang University, Email: {xxxshixiannva, liuyizhong, lby, liujianwei, hubin0205, linxun}@buaa.edu.cn

[†]A*STAR Institute for Infocomm Research, Email: lu_yuan@i2r.a-star.edu.sg

[‡]Nanyang Technological University, Email: tianwei.zhang@ntu.edu.sg

Abstract

Asynchronous complete secret sharing (ACSS) and asynchronous dynamic proactive secret sharing (ADPSS) are fundamental primitives for secret sharing and resharing in threshold systems. They serve broad applications in distributed key management (DKM), multi-party computation, and blockchain. However, ACSS constructions that employ homomorphic commitments incur notable computational overhead, especially in batched executions. Conversely, the lightweight variants require quadratic per-secret communication, which grows rapidly with the committee size. Building upon instances of ACSS, ADPSS constructions inevitably inherit these inefficiencies.

To address these limitations, we introduce GoSSamer, a concretely and asymptotically efficient protocol suite, where both GoSSamer-ACSS and GoSSamer-ADPSS achieve (1) lightweight computation with only finite-field arithmetic and hash functions, (2) asymptotically optimal, linear per-secret communication, (3) optimal resilience in asynchronous networks, and (4) post-quantum security. Leveraging our proposed share propagation and verification mechanisms, GoSSamer-ACSS reduces the runtime by 97% against the linear-communication scheme hbACSS (NDSS’22), and lowers the bandwidth usage by 75% compared to the lightweight solution SS24 (JoC’24) in a 64-node committee. By decoupling the ADPSS framework from the commitment-based ACSS through a new consistency verification technique, GoSSamer-ADPSS boosts key resharing throughput by $16\text{--}22\times$ compared to LongLive (Usenix Security’23), and reduces its bandwidth by 47.6% in 16-node committees. GoSSamer directly accelerates real-world applications like distributed key management. Specifically, the runtime in GoSSamer-ACSS-based distributed key generation outperforms DXK+23 (Usenix Security’23) by $7\text{--}11\times$, while GoSSamer-ADPSS-based key resharing outperforms LongLive by $9\text{--}13\times$.

1 Introduction

Threshold secret sharing [66] is a fundamental cryptographic primitive that distributes a secret among n parties such

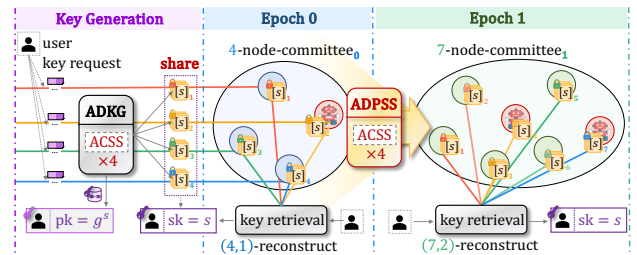


Figure 1: The lifecycle of key management in Web3Auth [62]. The private key sk is reconstructed upon dAPP sign-in request.

that any quorum can reconstruct it. This primitive has driven two critical extensions in the practical asynchronous network: asynchronous verifiable/complete secret sharing (AVSS/ACSS) and asynchronous dynamic proactive secret sharing (ADPSS) [23]. Operating without timing assumptions, the asynchronous model is immune to the potential failures under synchrony loss. In this setting, ACSS extends AVSS with *completeness*, where all honest participants eventually output their valid shares. Building upon ACSS, ADPSS *proactively* refreshes and migrates the secret shares across *dynamic* committees in the long-lived systems against corruption attacks, without revealing or altering the secret. Owing to their robustness and functionalities, ACSS and ADPSS are widely used in threshold cryptography [9, 36], multi-party computation (MPC) [68], and distributed key management (DKM) [62].

Real-world distributed key management systems [6, 62] leverage ACSS and ADPSS to facilitate decentralized and non-custodial control over users’ public and private key pairs. As shown in Figure 1, threshold key pairs are generated collaboratively among mutually distrustful nodes [36] through ACSS-based asynchronous distributed key generation [34] (ADKG), and periodically refreshed for the next epoch through ACSS-based ADPSS. The managed threshold key pairs power broad applications, from authentication platforms like Web3Auth [62] in MetaMask and wallet custody services like Lit Protocol [6], to cryptographic operations

Table 1: Comparison of relevant batched ACSS schemes; more analysis is given in Appendix A

Scheme	Lightweight ^b	Comm. Complexity [†]		Setup		Assumption	Post-quantum Security	Resilience
		(in total)	(# per secret)	no CRS?	no PKI?			
hbACSS [76]	✗	$O(\kappa bn + \kappa n^2 \log n)$	$O(\kappa n)$	✗	✗	q -SDH	✗	1/3
Bingo [1]	✗	$O(\kappa bn + \kappa n^2)$	$O(\kappa n)$	✗	✓	q -SDH	✗	
Haven++ [4]	✗	$O(\kappa bn + \kappa n^2 \log n)$	$O(\kappa n)$	✓	✓	DL + RO	✗	
SS24 [67]	✓	$O(\kappa bn^2 + \kappa n^3 \log n)$	$O(\kappa n^2)$	✓	✓	RO	✓	
Our Solution	✓	$O(\kappa bn + \kappa n^3 \log n)$	$O(\kappa n)$	✓	✓	RO	✓	1/3

^b **Lightweight** - We classify a protocol as lightweight if its computation is confined to lightweight cryptographic primitives such as hash functions.

[†] **Comm. Complexity** - Communication cost for all messages exchanged among honest nodes in the worst-case bound. Presented as: (1) *total complexity*: overall communication cost for all secrets; and (2) *per-secret complexity*: amortized cost per secret. Note that we denote n as the committee size, b the batch size, and κ the security parameter.

in Byzantine agreement and blockchain like threshold signature [33, 46] and threshold decryption [38, 69]. Consequently, the security and efficiency of ACSS and ADPSS are pivotal to the overall performance of such systems that rely on DKM.

1.1 Related Works

To maximize the utility of each protocol execution, recent ACSS schemes enable the sharing of a batch of secrets within a single ACSS instance. These variants are referred to as *batched* ACSS. Existing batched ACSS protocols focus on improving the communication and computation efficiency, as summarized in Table 1. Communicationally efficient batched ACSS schemes like hbACSS [76], Bingo [1], and Haven++ [4] optimize the per-secret communication complexity to the information-theoretic lower bound $O(\kappa n)$, where n is the number of participants and κ is the security parameter. These protocols, however, rely on additively homomorphic polynomial commitments [22, 57] for share verification, resulting in high computation and potential insecurity against quantum adversaries [9]. SS24 [67] proposes the first “*lightweight*”¹ and post-quantum secure ACSS protocol, which relies only on finite-field arithmetic and hash functions, yielding markedly lower computational overhead. Notably, our local benchmark reports that sharing 10^3 keys within a 31-node committee via SS24 [67] takes 11.7s of computation, compared to 368.8s in hbACSS [76] and 724.9s in Haven++ [4]. However, the per-secret communication complexity in the lightweight scheme SS24 [67] asymptotically degrades to $O(\kappa n^2)$.

Building upon ACSS, existing ADPSS constructions also reduce the asymptotic per-secret communication to the optimal $O(\kappa n)$ within each committee, as summarized in Table 2. Under Byzantine faults, the constructions [23, 78] incur a communication cost of $O(\kappa n^4)$ and $\exp(n)$, respectively. The *batched* ADPSS scheme, LongLive [77], innovates a batch-processing pattern and reduces the asymptotic per-secret communication complexity to the optimal $O(\kappa n)$ within each committee. However, the cross-committee communication

¹ Following SS24 [67], we classify a protocol as “lightweight” if it builds upon lightweight cryptographic primitives such as hash functions.

Table 2: Comparison of relevant ADPSS schemes

Scheme ^b	Lightweight	Comm. Complexity [†]		Dynamic	Setup no CRS?
		(inner-com)	(cross-com)		
CKL+02 [23]	✗	$O(\kappa n^4)$	$O(\kappa n^4)$	✗	✗
ZSR05 [78]	✗	$\exp(n)$	$\exp(n)$	✓	✓
LongLive ¹ [77]	✗	$O(\kappa n^2)$	$O(\kappa n^2)$	✓	✓
LongLive ² [77]	✗	$O(\kappa n)$	$O(\kappa n^2)$	✓	✗
Our Solution	✓	$O(\kappa n)$	$O(\kappa n)$	✓	✓

^b **Scheme** - LongLive¹ [77] builds upon Pedersen commitment, while LongLive² [77] builds upon KZG commitment [57]. Appendix A presents more analysis of ADPSS.

[†] **Comm. Complexity** - The per-secret communication cost for all messages exchanged among all honest nodes in the worst-case bound. Presented as (1) inner-committee communication complexity; and (2) cross-committee communication complexity.

complexity in LongLive [77] is $O(\kappa n^2)$. Beyond communication cost, the computational overhead in ADPSS remains high, as existing constructions [23, 77, 78] are tightly coupled to the computationally expensive commitment-based ACSS, from which the commitments are derived for further usage in ADPSS’s verification procedures. Specifically, in distributed key management, refreshing keys for 10^4 users across 28-node committees with LongLive [77] takes 720s of computation, dominated by the underlying commitment-based ACSS.

Thus, the above issues expose an open question:

Can we design efficient ADPSS and ACSS protocols that simultaneously achieve (i) “*lightweight*” computation and (ii) *optimal per-secret communication* of $O(\kappa n)$?

1.2 Our Contributions

We answer the question by firstly presenting a batched ACSS scheme, GoSSamer-ACSS. Building upon it, we further construct a batched ADPSS protocol, GoSSamer-ADPSS.

• **GoSSamer-ACSS: a lightweight batched ACSS scheme with linear communication.** Our design introduces two core techniques: a novel *batched original-evaluation propagation* (B-OEP) paradigm that strategically organizes share distribution, and a *bivariate-polynomial-based degree checking* (BP-DC) for lightweight share verification. Our B-OEP mechanism reduces the transmitted message size in lightweight

ACSS scheme [67] by a factor of $O(n)$, without applying any transformation to the original evaluations sent from dealer. This design eliminates the structural reliance on the additively homomorphic commitments in prior linear-communication ACSS constructions [1, 4, 76], and enables lightweight share verification through our BP-DC technique. Consequently, GoSSamer-ACSS is the first concretely efficient scheme to simultaneously offer lightweight computation and linear per-secret communication. Its high batch-processing efficiency is particularly suitable for large scale deployments like MPC, DKM, and random beacon.

- **GoSSamer-ADPSS: a lightweight batched ADPSS Protocol with linear communication.** Our solution introduces two key mechanisms: (1) A lightweight *dual-committee sharing consistency verification* mechanism, which decouples commitment-based ACSS schemes from ADPSS, thereby establishing GoSSamer-ADPSS as a general framework capable of supporting lightweight ACSS schemes. This design directly unlocks a lightweight ADPSS instantiation that builds upon our lightweight GoSSamer-ACSS, yielding a boost in throughput. (2) A *dual-committee reconstruction* technique, which reduces the cross-committee communication complexity to the optimal $O(\kappa n)$ per secret. These properties enable GoSSamer-ADPSS to support large-scale share refreshing, especially in MPC, DKM, and blockchain systems.

- **Implementation and evaluation.** To demonstrate the concrete efficiency of our solutions, we implement GoSSamer-ACSS, GoSSamer-ADPSS, and the lightweight ACSS scheme SS24 [67]. We also reproduce protocols hbACSS [76], Haven++ [4], and LongLive [77]. The evaluation results report that compared to other ACSS schemes with the same linear-communication, hbACSS [76] and Haven++ [4], GoSSamer-ACSS achieves a $15\text{-}63\times$ improvement in throughput; compared to the lightweight solution SS24 [67], GoSSamer-ACSS reduces the bandwidth by 75.2% at $n = 64$. Moreover, GoSSamer-ADPSS achieves $14\text{-}26\times$ more throughput compared to LongLive [77], and a 55.8% reduction in bandwidth at $n = 31$. When deployed in a distributed key management system with $n = 10$, the GoSSamer-ACSS-based distributed key generation [50] outperforms DXK+23 [34] by $7\times$ in runtime, while GoSSamer-ADPSS-based key resharing processes 1900 additional keys per second compared to LongLive [77].

2 Preliminaries and Problem Formulation

2.1 Notation

Let b denote the output batch size. We use com to represent a committee, where n is the total number of participating nodes, and t is the threshold, which also corresponds to the maximum number of corrupted nodes tolerated. Furthermore, we use $:=$ for deterministic assignment, $\leftarrow$ for probabilistic assignment, and $\parallel$ for vector concatenation. Bold symbols such as $\mathbf{a}$ denote vectors. The element-wise (n, t) -secret sharing of a vector

$\mathbf{a}$ is written as $[\mathbf{a}]$, where $[\mathbf{a}]_i$ denotes the share for the i -th node. For two m -dimensional vectors $\mathbf{a}, \mathbf{b}$, we define their inner product as $\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^m a_i b_i$. Let $\mathbf{f}(x)$ denote a vector of polynomials over $\mathbb{Z}_p[x]$. For any integer i , $\mathbf{f}(i)$ consists of the evaluations at $x = i$ of all component polynomials in $\mathbf{f}(x)$.

2.2 System Model

We consider a distributed system designed for the long-term secure management of secrets. Intuitively, the system's operation is structured into discrete protocol execution periods termed *epochs*. We adopt the rigorous definition of asynchronous epoch from Schultz-MPSS [64] and LongLive [77] without timing assumptions, which is further elaborated in Appendix B. For each epoch, a committee is defined, consisting of a set of nodes responsible for managing the shares of the secrets. Each node is assigned a unique identity for its role during that epoch, e.g., P_i for the i -th node. The composition of the committee may change across epochs, with the relationship between successive committees ranging from overlap to complete disjointness. If a node participates in the committees of two epochs, it will operate under distinct identities.

Adversary model. Our adversary model follows that of LongLive [77] by considering a static, probabilistic polynomial-time (P.P.T.) adversary $\mathcal{A}^2$. In each epoch, a committee of n nodes is responsible for executing the protocol, and the adversary may statically corrupt up to t of them before the epoch begins, where $t < n/3$. Once corrupted, a node is fully controlled by the adversary, who can read all its inputs intended for the current epoch and determine the messages it sends. During committee transitions, any node involved in secret resharing is considered to belong to both epochs simultaneously. The corruption of such a node is counted in the adversary's budget in both epochs. In static committees, this adversary model is equivalent to a mobile adversary, which may eventually corrupt all nodes over the system lifetime but is constrained to corrupting at most t nodes in any given epoch.

Network model. Communication occurs over an *asynchronous* network of authenticated point-to-point channels between any two nodes. The adversary can arbitrarily delay and reorder the messages on any channel. However, messages transmitted between nodes are guaranteed to be eventually delivered.

2.3 Building Blocks

Robust Polynomial Interpolation. Given $n = 3t + 1$ evaluations of an unknown t -degree polynomial, of which at most t may be adversarially corrupted, robust polynomial interpolation, typically instantiated by Gao's decoder [43] or the Welch-Berlekamp decoder [71], uniquely reconstructs the polynomial under this error bound. In asynchronous settings, an

²When model the hash function as a quantum random oracle [19], the adversary can also access a quantum computer.

online error-correction variant [13] incrementally updates the candidate polynomial as each new evaluation arrives, thereby eliminating the indistinguishability of whether a missing share is delayed or intentionally withheld by the adversary.

Secure message distribution (SMD). Secure message distribution is a lightweight message distribution protocol originally proposed in SS24 [67]. At a high level, SMD follows the disperse-retrieve paradigm similar to the asynchronous verifiable information dispersal (AVID) [25], which is typically employed in asynchronous protocols [45, 76]. SMD provides two primitives: (i) distribute, where the dealer distributes message $\mathbf{m} = (m_1, \dots, m_n)$ to a committee with n nodes, such that each node $\mathcal{P}_i$ receives the message m_i ; (ii) forward, where a node $\mathcal{P}_i$ can optionally forward a message m_u received either in the distribute phase or in a prior forward phase, to another node $\mathcal{P}_j$ in the committee. Specifically, the dealer commits all messages through the Merkle-tree and symmetrically encrypts them. Upon forwarding, the Merkle path authenticates the origin of each message. The communication complexity of the distribute phase is $O(|\mathbf{m}| + \kappa n^2 \log n)$, while the forward phase requires $O(|m_i| + \kappa n \log n)$ communication to forward a message m_i . An (n, t) -SMD protocol satisfies the following properties if whenever at most t -out-of- n nodes are corrupted:

- **Correctness:** If any honest node $\mathcal{P}_i$ outputs a message in either the distribute or forward phase, the output must be consistent with a vector $\mathbf{m} = (m_1, \dots, m_n)$ in which each message m_j is output by node $\mathcal{P}_j$ during the distribute phase. In the forward phase, any message output by an honest node that is claimed to originate from $\mathcal{P}_u$ must be m_u . Moreover, if the dealer is honest, then $\mathbf{m}$ must equal the message vector originally input by the dealer.
- **Secrecy:** If both the dealer and node $\mathcal{P}_i$ are honest, and no honest node forwards m_i to any corrupted party, then the adversary learns nothing about m_i .
- **Completeness:** In the distribute phase, if the dealer is honest or any honest node outputs a message, all honest nodes output eventually. If an honest node forwards a message m_u to another honest node $\mathcal{P}_i$, then $\mathcal{P}_i$ outputs m_u eventually.

SMD without *secrecy* can be instantiated using AVID. Adding secrecy to AVID is conventionally done through public-key cryptography. For “lightweight” computation, SMD instead utilizes symmetric encryption and designs a symmetric key distribution phase to avoid the secure channel assumption.

Next, we present the functionalities of three components that are commonly used in asynchronous protocols [34, 36, 76, 77]; definitions of their properties are given in Appendix C.

Reliable broadcast. Reliable broadcast ensures that a message m distributed by a dealer $\mathcal{P}$ is consistently received by all n participants. We adopt the construction from DXR21 [35] with a communication complexity of $O(n|m| + \kappa n^2)$.

Multi-valued validated asynchronous byzantine agreement (MVBA). MVBA is an asynchronous Byzantine fault-tolerant

consensus protocol, initially proposed in CKPS01 [24]. In MVBA, each participating node proposes an input value, and the protocol ensures agreement on one of these values that satisfies a predefined common predicate. The adopted MVBA construction is further elaborated in Section 8.

One-sided voting (OSV). One-sided voting is a weak agreement primitive in which each node independently decides whether to endorse a given event, and the participation of the protocol is counted as a vote. A successful decision is reached once at least $n - t$ nodes honestly participate. OSV is instantiated using the Bracha broadcast [21], omitting the proposal phase. This protocol terminates in $O(1)$ rounds and incurs a communication complexity of $O(\kappa n^2)$.

2.4 Threshold Secret Sharing Protocols

ACSS: (batched) asynchronous complete secret sharing.

An (n, t, b) -ACSS protocol allows a dealer $\mathcal{P}$ to share a batch of b secrets $\mathbf{a} \in \mathbb{Z}_p^b$ among a committee of nodes $\mathcal{P}_1, \dots, \mathcal{P}_n$, such that each node $\mathcal{P}_i$ receives a share $[\mathbf{a}]_i$, where $\mathbf{a} = (a_1, \dots, a_b)$. The secret vector $\mathbf{a}$ can be reconstructed from the shares of $t + 1$ nodes using Lagrange interpolation. Any subset of at most t shares reveals no information about $\mathbf{a}$. The protocol is said to be batched if $b > 1$. In such settings, we assume that $b = \text{poly}(n)$, as considered in prior work [67, 77]. An ACSS protocol satisfies the following properties:

- **Correctness:** If the dealer $\mathcal{P}$ is honest and inputs the secret vector $\mathbf{a}$, then there exists a batch of b polynomials $\mathbf{f}(x) \in \mathbb{Z}_p[x]_{\leq t}^b$, each of degree at most t , such that $\mathbf{f}(0) = \mathbf{a}$ and every honest node $\mathcal{P}_i$ outputs the share $[\mathbf{a}]_i = \mathbf{f}(i)$.
- **Completeness:** If any honest node outputs, then there exists a vector of polynomials $\tilde{\mathbf{f}}(x) \in \mathbb{Z}_p[x]_{\leq t}^b$, such that every honest node $\mathcal{P}_i$ eventually outputs $\tilde{\mathbf{f}}(i)$.
- **Secrecy:** If the dealer is honest, the protocol reveals nothing about $\mathbf{a}$ to the adversary. Formally, there exists a P.P.T simulator $\mathcal{S}_{\text{acss}}$ which, given the shares of corrupted nodes and a vector of independently sampled secrets, can produce a simulated view in the ideal world, such that the ideal and real worlds are indistinguishable from the view of $\mathcal{A}$.³

Applications such as MPC, ADKG, and ADPSS critically rely on *completeness*, to prevent a malicious dealer from sending invalid shares to some honest nodes in a successful sharing instance and thus affecting subsequent computations.

ADPSS: (batched) asynchronous dynamic proactive secret sharing. An (n, t, n', t', b) -ADPSS protocol facilitates the *proactive* resharing of b secrets $\mathbf{s} \in \mathbb{Z}_p^b$ from an old committee com in epoch $(e - 1)$ to a *dynamically* reconfigured new committee com' in epoch e . Specifically, the old committee consists of n nodes $(\mathcal{P}_1, \dots, \mathcal{P}_n)$, where each node $\mathcal{P}_i$ holds an (n, t) -threshold share $[\mathbf{s}]_i$ of $\mathbf{s}$. The protocol enables the nodes in the old committee to collaboratively reshare the secrets

³The adversary’s view includes all messages sent from honest nodes.

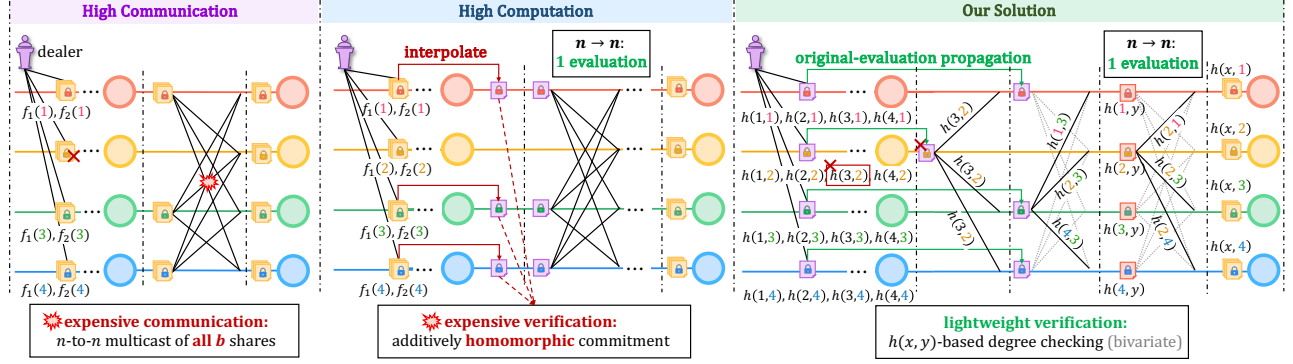


Figure 2: Technical overview of GoSSamer-ACSS with batch size $b = 2$ and committee size $n = 4$ for simplicity.

to the new committee of n' nodes, such that each node $\mathcal{P}_i'$ obtains an (n', t') -threshold share $[\bar{s}]_i$ of the same secrets $\mathbf{s}$. An ADPSS protocol satisfies the following properties:

- **Correctness:** Suppose there exists a vector of t -degree polynomials $\mathbf{f}(x)$ with $\mathbf{f}(0) = \mathbf{s}$, and each node $\mathcal{P}_i$ in the old committee holds the share $[\mathbf{s}]_i = \mathbf{f}(i)$. If an honest node $\mathcal{P}_i'$ in the new committee com' outputs its share $[\bar{s}]_i$, then all honest nodes in com' output their shares eventually, and there exists a vector of t' -degree polynomials $\bar{\mathbf{f}}(x)$ such that $\bar{\mathbf{f}}(0) = \mathbf{s}$ and the share $[\bar{s}]_i = \bar{\mathbf{f}}(i)$ for each honest node $\mathcal{P}_i'$.
- **Termination:** If all honest nodes in com initiate the protocol, all honest nodes in com' output eventually.
- **Secrecy:** No information about $\mathbf{s}$ is revealed to the adversary $\mathcal{A}$. Formally, there exists a P.P.T simulator $\mathcal{S}_{\text{adpss}}$ which, given the shares of corrupted nodes in the old committee and a vector of independently sampled secrets, can produce a simulated view in the ideal world such that the adversary's view in the ideal and real world is indistinguishable.

3 Technical Contribution Overview

3.1 GoSSamer-ACSS

Motivation. Existing batched ACSS protocols [1, 4, 67, 76] are driven by two objectives: minimizing communication and computation costs. To illustrate this, consider a toy verifiable secret sharing example where the dealer encodes each secret as $f(0)$ in a t -degree polynomial $f(x)$ and distributes the evaluation $f(i)$ to each node i as its secret share. In asynchronous networks, *complete* secret sharing further requires that all nodes receive their valid shares, even if the dealer misbehaves. Current solutions fall into two paradigms. (1) *Computational efficient:* The lightweight protocol SS24 [67] achieves high computational efficiency with only lightweight cryptographic primitives. However, to ensure completeness, all n nodes multicast all their shares to all nodes, resulting in $O(\kappa n^2)$ per-secret communication, as shown in Figure 2, left. (2) *Communicational efficient:* Protocols like hbACSS [76],

Bingo [1], and Haven++ [4] achieve completeness with the optimal $O(\kappa n)$ per-secret communication by letting each recipient node forward the interpolated evaluations, as shown in Figure 2, middle. However, verifying them requires expensive additively homomorphic polynomial commitments, e.g., KZG [57] or Pedersen commitment.

Above situations become increasingly problematic as the systems scale: lightweight schemes cannot scale to large networks, while communication-optimal schemes cannot scale to large batches. GoSSamer-ACSS resolves this by establishing *a new evaluation propagation paradigm B-OEP* and *a new share verification mechanism BP-DC*, as in Figure 2, right.

B-OEP: linear-communication original-evaluation forwarding. The key insight in our batched original-evaluation propagation is letting each node *forward the original polynomial evaluations* sent from the dealer, to avoid the reliance on homomorphic commitments, while attaining the message size of $O(\kappa b/n)$ in the n -to- n multicast for the optimal linear per-secret communication. Specifically, for a batch of $b = t + 1$ secrets, the dealer samples a (t, t) -degree bivariate polynomial $h(x, y)$, where $h(x, 0)$ is interpolated from the $t + 1$ secrets, and thus each row polynomial $h(x, i)$ is the share for node i . Instead of distributing the polynomial $h(x, i)$ in the coefficient form to node i directly, the dealer sends it in the *evaluation form*: $h(1, i), \dots, h(n, i)$. When the dealer misbehaves, the share for a node i may be invalid. We let node i disclose this invalidity by *forwarding an invalid original evaluation* $h(e, i)$ to all nodes. Thus, a malicious dealer can be confirmed, and the recovery process is executed for those nodes with invalid shares to ensure completeness. In this process, each node i with valid evaluations $h(1, i), \dots, h(n, i)$ forwards *each original evaluation* $h(j, i)$ to each node j . Each recipient j can then verify and interpolate for the polynomial $h(j, y)$, and distribute each $h(j, k)$ to each node k , allowing node k to *robustly reconstruct* its row polynomial $h(x, k)$ that encodes all its shares.

This pattern yields a communication complexity of $O(\kappa n)$ per secret, but avoids creating any interpolated values that would need additively homomorphic commitments for verification. Moreover, these non-interpolated values in B-OEP

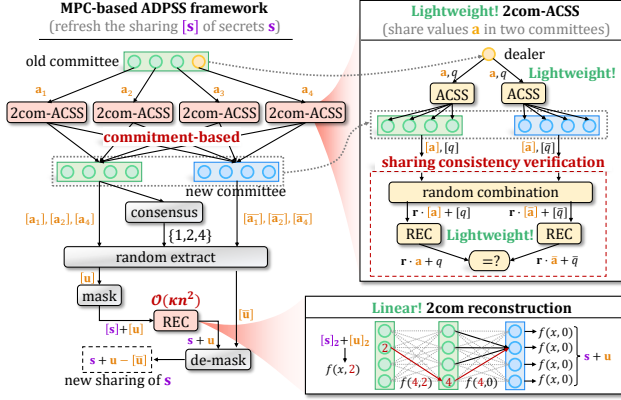


Figure 3: Technical overview of GoSSamer-ADPSS. “2com” is an abbreviation for “dual-committee.”

enable the lightweight verification mechanism presented next.

BP-DC: lightweight verification for bivariate polynomial evaluations. In B-OEP, each original evaluation must be verified in the following two aspects:

- *Evaluation correctness:* Each evaluation $h(j, i)$ must lie on the (t, t) -degree secret sharing polynomial $h(x, y)$.
- *Evaluation authenticity:* In the process of disclosing or recovering, the forwarded evaluation, $h(j, i)$, must be originally sent from the dealer, rather than faked by the recipients.

For evaluation correctness, we introduce BP-DC, a bivariate-polynomial-based degree checking technique. Inspired by classical MPC [52], BP-DC proves that each evaluation $h(j, i)$ lies on the bivariate polynomial $h(x, y)$ with only finite-field arithmetic. At a high level, BP-DC constructs a lightweight verification script $v(x, y) = r \cdot h(x, y) + g(x, y)$ with a randomly sampled challenge value r and a randomly sampled polynomial $g(x, y)$. Guaranteed by the Schwartz-Zippel lemma (proved in Appendix E), if the evaluation $v(j, i)$ lies on the script $v(x, y)$, then $h(j, i)$ lies on $h(x, y)$, except with negligible probability. Thus, after the dealer broadcasts the script $v(x, y)$, the correctness of each evaluation $h(j, i)$ can be verified by letting each recipient locally compute $v(j, i)$ and check if it lies on the script $v(x, y)$. For evaluation authenticity, we utilize the secure message distribution protocol, which facilitates a lightweight authentication mechanism through the Merkle-tree, for authenticating the evaluations explicitly published by the dealer. Since B-OEP forwards only the original evaluations sent from dealer, their authenticity can be naturally verified by the lightweight secure message distribution protocol.

3.2 GoSSamer-ADPSS

Motivation. Existing ADPSS protocols [41, 53, 54, 73, 77, 78] are built on *commitment-based* ACSS. The polynomial commitments published in these ACSS schemes are later reused to prove the correctness of the resharing process, which

couple ADPSS tightly to heavy cryptographic primitives and blocks the adoption of our lightweight ACSS. Moreover, existing ADPSS designs [41, 53, 54, 73, 78] incur at least quadratic cross-committee communication. Among them, although the best-known result in LongLive [77] achieves a per-secret communication of $O(\kappa n)$ within each committee, it still involves an n -to- n multicast across committees for each secret, leading to the $O(\kappa n^2)$ per-secret communication, which is further evidenced by our bandwidth evaluation in Section 6.

With this state of affairs, we propose GoSSamer-ADPSS, as shown in Figure 3. GoSSamer-ADPSS adopts the MPC-based ADPSS framework as in [41, 53, 77]. To decouple the framework from polynomial commitments, we design a *lightweight dual-committee ACSS with a new sharing consistency verification mechanism at its core*. Moreover, to optimize the cross-committee communication, we design a *new linear-communication dual-committee reconstruction paradigm*.

We begin by briefly recalling the MPC-based ADPSS framework. First, the old committee distributively generates a random sharing $[u]$, and the new committee also distributively generates a random sharing $[\bar{u}]$, such that $u = \bar{u}$. Next, possessing the sharing $[s]$ of the secrets to be reshared, the old committee reconstructs $s + u$, and multicasts it to the new committee. The new committee then computes the new sharing as $[\bar{s}] := (s + u) - [\bar{u}]$. We now present our refinements that yield a lightweight and linear communication construction.

Lightweight dual-committee sharing consistency verification. In the MPC-based framework, distributively generating $[u]$ within a single committee is typically done by a share-consensus-extract paradigm [34, 41, 53, 77], where each node shares some random values a through ACSS, followed by a consensus and a random extraction phase to obtain the random sharing $[u]$. However, in the dual-committee setting, existing solutions prove $u = \bar{u}$ through the polynomial commitments derived in the underlying expensive commitment-based ACSS.

To resolve this, we propose a lightweight dual-committee ACSS, where the dealer shares a to the old and new committees respectively through the lightweight GoSSamer-ACSS, yielding a sharing $[a]$ in the old committee and $[\bar{a}]$ in the new committee. Next, all nodes invoke our newly devised lightweight sharing consistency verification mechanism to confirm $a = \bar{a}$, which is inspired by a classical MPC technique [52]. It operates by having the two committees check the equality of a randomized linear combination. Specifically, the dealer shares an additional random value q to the old and new committees, respectively. Given a random challenge vector r , when the underlying secrets (a, q) and $(\bar{a}, \bar{q})$ differ, the challenge vector r makes $r \cdot a + q$ and $r \cdot \bar{a} + \bar{q}$ coincide with a probability of at most $1/|\mathbb{Z}_p|$. Thus, if the robust interpolation results of $r \cdot [a] + [q]$ and $r \cdot [\bar{a}] + [\bar{q}]$ are the same, then $[a]$ and $[\bar{a}]$ are the sharings of the same secrets a , except with negligible probability. Therefore, observing the robust interpolation results allows all participants to confirm the consistency of the sharing across committees.

Linear-communication dual-committee reconstruction. In the MPC-based framework, each node in the old committee multicasts all the b secrets $\mathbf{s} + \mathbf{u}$ to all nodes in the new committee, leading to a per-secret communication of $O(\kappa n^2)$. To facilitate this process with optimal linear per-secret communication and lightweight primitives, we propose the following dual-committee reconstruction technique.

The key idea in the dual-committee reconstruction technique is to reduce the cross-committee message size to $O(\kappa b/n)$, compared to $O(\kappa b)$ in the existing MPC-based framework [41, 53, 77]. Specifically, possessing a vector of b shares $[\mathbf{s} + \mathbf{u}]_i$, each old committee node i interpolates each subset of $(t + 1)$ shares in $[\mathbf{s} + \mathbf{u}]_i$ into a t -degree polynomial $f(x, i)$. It then distributes each evaluation $f(j, i)$ to each node j in the old committee, where node j can collect enough evaluations and run the robust polynomial interpolation to get the column polynomial $f(j, y)$. As the values $\mathbf{s} + \mathbf{u}$ are encoded at $f(x, 0)$, we let each old committee node j with $f(j, y)$ multicast the evaluation $f(j, 0)$ to the new committee. Each new committee node can then run robust interpolation to get $f(x, 0)$ and decode it for $\mathbf{s} + \mathbf{u}$. As a result, the cross-committee message size is $O(\kappa)$ for resharing $(t + 1)$ secrets, thus the per-secret communication in the n -to- n multicast is reduced to $O(\kappa n)$.

4 GoSSamer-ACSS

GoSSamer-ACSS is a batched ACSS (defined in Section 2.4) scheme that simultaneously shares a batch of secrets $\mathbf{a}$ among a committee. Its public input includes the field $\mathbb{Z}_p$ of prime order p , the number of participating nodes n , threshold t and batch size b . The dealer privately inputs secrets $\mathbf{a}$ into the protocol. If the protocol outputs, then it outputs the (n, t) -threshold share $[\mathbf{a}]_i$ to each node $\mathcal{P}_i$, for all $i \in [n]$. The theoretical analysis of GoSSamer-ACSS is given in Appendix E.

Initially, secret shares are distributed by the dealer to all participating nodes, along with a lightweight verification script (Phase 1). Upon successfully verifying its share, each node casts a vote to endorse the sharing instance, which is considered successful if at least $n - 2t$ honest nodes obtain its valid share (Phase 2). Nodes that fail to obtain valid shares disclose evidence of the dealer's misbehavior. A confirmed malicious dealer implies secret leakage (Phase 3). Finally, to ensure completeness, nodes forward their valid shares to assist in recovering shares for nodes without them (Phase 4).

4.1 Concrete Scheme

Phase 1 : lightweight secret sharing.

Algorithm 1 illustrates the procedure of lightweight secret sharing in Phase 1. The dealer begins by partitioning the b input secrets into groups of size $t + 1$. For each group, it interpolates the secrets into a t -degree univariate polynomial. Each such polynomial is then embedded into a randomly sampled (t, t) -

Algorithm 1 GoSSamer-ACSS

public input: field $\mathbb{Z}_p$, committee size n, threshold t, batch size b private input: secret vector $\mathbf{a}$ of length b (as dealer $\mathcal{P}$) output: share $[\mathbf{a}]_i$ for each participating node $\mathcal{P}_i$ Phase 1 : lightweight secret sharing (as dealer $\mathcal{P}$) 1: • divide $\mathbf{a}$ into $m = b/(t + 1)$ groups $(\mathbf{a}_1, \dots, \mathbf{a}_m)$ of size $t + 1$ 2: for each group $\mathbf{a}_j$ do ▶ encode $\mathbf{a}$ in $\mathbf{A}(x, 0)$ 3: ◦ interpolate all entries in $\mathbf{a}_j$ into a t-degree polynomial $f(x)$ 4: ◦ sample a (t, t)-degree polynomial $A_j(x, y)$ with $A_j(x, 0) := f(x)$ 5: • let $\mathbf{A}(x, y) := (A_1(x, y), \dots, A_m(x, y))$ 6: ▶ $\mathbf{A}(x, y)$ is the secret sharing polynomial of $\mathbf{a}$ 7: • sample a (t, t)-degree blinding polynomial $E(x, y)$ 8: for $k \in [n]$ do ▶ distribute $\mathbf{A}(x, j)$ to $\mathcal{P}_j$ in the evaluation form 9: ◦ distribute $(\mathbf{A}(k, j), E(k, j))$ to node $\mathcal{P}_j$ through SMD, $j \in [n]$ 10: • extract the generated Merkle root $\mathbf{rt}$ during distribution 11: • calculate the m-dimension challenge $\mathbf{r}$ through $\mathbf{rt}$ 12: • reliable broadcast $v(x, y) := \mathbf{r} \cdot \mathbf{A}(x, y) + E(x, y)$ to all nodes Phase 2 : lightweight share verification (as node $\mathcal{P}_i$) 13: upon receiving $(\mathbf{A}(k, i), E(k, i), \mathbf{rt})$ for all $k \in [n]$ do 14: • calculate the challenge vector $\mathbf{r}$ (as in Lines 10-11) 15: upon outputting $v(x, y)$ from reliable broadcast do 16: if $v(k, i) = \mathbf{r} \cdot \mathbf{A}(k, i) + E(k, i)$ for all $k \in [n]$ do 17: ◦ state.set(happy) and init OSV ▶ valid share, vote 18: else locally store a faulty pair $(\mathbf{A}(e, i), E(e, i))$ 19: upon OSV output do 20: if state.get = happy do ▶ $n - t$ nodes happy, ACSS succeeds 21: • interpolate the first $t + 1$ evaluations into $\mathbf{A}(x, i)$ 22: • decode $\mathbf{A}(x, i)$ into $[\mathbf{a}]_i$ and output ▶ output valid shares 23: else state.set(complain) ▶ begin complain
--

degree bivariate polynomial, as the row at $y = 0$. This process results in a vector of bivariate polynomials $\mathbf{A}(x, y)$, where the row polynomial $\mathbf{A}(x, 0)$ correspond to the encoding of the b secrets and $\mathbf{A}(x, i)$ encodes the share of node $\mathcal{P}_i$ (Lines 1-6).

Next, the dealer distributes each share-encoding polynomial $\mathbf{A}(x, i)$ to each node $\mathcal{P}_i$ through the following steps: The dealer first samples a random (t, t) -degree blinding polynomial $E(x, y)$ (Line 7), which is used for blinding the subsequent lightweight verification script. It then distributes both the share $\mathbf{A}(x, i)$ and the blinding share $E(x, i)$ to each node $\mathcal{P}_i$ in the evaluation form. Specifically, the dealer sends to each node $\mathcal{P}_i$ the evaluations $(\mathbf{A}(1, i), E(1, i)), \dots, (\mathbf{A}(n, i), E(n, i))$, through the distribute function in SMD, the secure message distribution protocol defined in Section 2.4 (Lines 8-9).

Finally, the dealer generates a lightweight verification script via the following steps, enabling the recipient nodes to validate their received evaluations: it first derives a challenge vector $\mathbf{r}$ matching the length of $\mathbf{A}(x, y)$ (Lines 10-11), and computes the verification script through equation $v(x, y) := \mathbf{r} \cdot \mathbf{A}(x, y) + E(x, y)$, which is further reliably broadcast to all nodes (Line 12). Here, the blinding polynomial $E(x, y)$ serves to conceal the distribution of $\mathbf{A}(x, y)$.

Challenge vector derivation. Recall the BP-DC technique elaborated in Section 3.1. To verify that $\mathbf{A}(j, i)$ equals $\mathbf{A}(x, y)$ evaluated at (j, i) , each recipient node locally computes $v(j, i) := \mathbf{r} \cdot \mathbf{A}(j, i) + E(j, i)$. It then checks whether the re-

Algorithm 2 GoSSamer-ACSS (handle dealer misbehavior)

Phase ③ : lightweight dealer disclosure (as node $\mathcal{P}_i$)

- 1: **upon** state.get(complain) **do**
- 2: • locally retrieve the faulty pair $(\mathbf{A}(e, i), E(e, i))$
- 3: • forward (complain, $\mathbf{A}(e, i), E(e, i)$) to all nodes through SMD
- 4: ▶ disclose the evidence of dealer misbehavior
- 5: **upon** outputting (complain, $\mathbf{A}(e, k), E(e, k)$) from $\mathcal{P}_k$ in SMD **do**
- 6: ▶ SMD ensures the message initially comes from the ACSS dealer
- 7: **if** $v(e, k) \neq \mathbf{r} \cdot \mathbf{A}(e, k) + E(e, k)$ **do**
- 8: • enter the recovery phase ▶ the malicious dealer is confirmed

Phase ④ : lightweight share recovery (as node $\mathcal{P}_i$)

- 9: **if** state.get(happy) and hasn't sent rec message **do**
- 10: **for** $l \in [n]$ **do** forward (rec, $\mathbf{A}(l, i), E(l, i)$) to $\mathcal{P}_l$ through SMD
- 11: **upon** outputting (rec, $\mathbf{A}(i, j), E(i, j)$) from $\mathcal{P}_j$ in SMD **do**
- 12: **if** $v(i, j) = \mathbf{r} \cdot \mathbf{A}(i, j) + E(i, j)$ **do**
- 13: • $\mathcal{Q}_{\text{val}} := \mathcal{Q}_{\text{val}} \cup (j, \mathbf{A}(i, j))$
- 14: **if** $\mathcal{Q}_{\text{val}}$ reaches the size $(t + 1)$ **do**
- 15: ◦ interpolate $\mathcal{Q}_{\text{val}}$ for $\mathbf{A}(i, y)$ ▶ all honest node $\mathcal{P}_i$ get $\mathbf{A}(i, y)$
- 16: ◦ send $\mathbf{A}(i, k)$ to $\mathcal{P}_k$ for all $k \in [n]$
- 17: **upon** receiving $\mathbf{A}(j, i)$ from node $\mathcal{P}_j$ **do**
- 18: • add $(j, \mathbf{A}(j, i))$ into set $\mathcal{Q}_{\text{rec}}$
- 19: • run robust interpolation when the size of $\mathcal{Q}_{\text{rec}}$ reaches $(n - t)$
- 20: **upon** outputting $\mathbf{A}(x, i)$ from robust interpolation **do**
- 21: • decode $\mathbf{A}(x, i)$ into $[\mathbf{a}]_i$ and output

sulting value $v(j, i)$ lies on the verification script $v(x, y)$. This test requires the dealer and all recipient nodes to share the same, randomly sampled challenge vector $\mathbf{r}$, and it is usually generated through the interactive challenge–response [50, 67]. In a distributed setting, the parties would invoke random beacon [9] for randomness generation. To avoid this extra cost, we apply the Fiat-Shamir transformation: both the dealer and the recipients compute the hash of $\mathbf{A}(x, y)$'s lightweight commitment, namely the Merkle root generated in the secure message distribution (Lines 10-11). Since the root is bound to all evaluations $\{\mathbf{A}(j, i)\}_{j, i \in [n]}$, the hash output yields a random yet identical vector for every party.

Phase ②: lightweight share verification.

Algorithm 1 also illustrates the procedure of lightweight share verification in Phase 2. In this phase, each node verifies the received evaluations. First, all participating nodes derive the same challenge vector $\mathbf{r}$ as the dealer (Lines 13-14). Next, each node $\mathcal{P}_i$ waits for the dealer to reliably broadcast the verification script $v(x, y)$ (Line 15). It then locally computes $\{v(k, i)\}_{k \in [n]}$, and checks whether each result equals the evaluation of $v(x, y)$ at (k, i) (Line 16). If the verification succeeds, the node sets its local state to happy and initiates the one-sided voting protocol (Line 17). Otherwise, it locally stores a faulty evaluation pair $(\mathbf{A}(e, i), E(e, i))$ that fails the verification (Line 18). When the one-sided voting protocol outputs a signal, at least $n - 2t$ honest nodes hold valid evaluations, then each happy node $\mathcal{P}_i$ interpolates $\mathbf{A}(1, i), \dots, \mathbf{A}(t + 1, i)$ into $\mathbf{A}(x, i)$, and decodes it into its secret share $[\mathbf{a}]_i$ by inverting the encoding process described in Lines 1-5 (Lines 19-22). Any node not in the happy state then sets its local state to

complain (Line 23), and proceeds to the subsequent phases.

Phase ③: lightweight dealer disclosure.

Algorithm 2 illustrates the procedure of lightweight dealer disclosure in Phase 3. In this phase, each node in the complain state discloses the evidence of dealer misbehavior to all nodes. Specifically, the complaining node $\mathcal{P}_i$ locally retrieves the stored faulty evaluation pair $(\mathbf{A}(e, i), E(e, i))$ (Lines 1-2). The faulty pair is then forwarded to all the other nodes as a complain message using the secure message distribution, to inform this invalidity (Lines 3-4). Upon receiving a complain message from a node $\mathcal{P}_k$, each node $\mathcal{P}_i$ locally verifies the correctness of the evaluation pair by calculating $\tilde{v}(e, k) := \mathbf{r} \cdot \mathbf{A}(e, k) + E(e, k)$. If $\tilde{v}(e, k)$ doesn't equal the evaluation at (e, k) on the lightweight verification script $v(x, y)$, the evaluation pair sent from $\mathcal{P}_k$ is indeed invalid, indicating the dealer's misbehavior. As a result, node i enters the recovery phase (Lines 5-8).

Phase ④: lightweight share recovery.

Algorithm 2 also illustrates the procedure of lightweight share recovery in Phase 4. In this phase, a malicious dealer is confirmed and secrecy is no longer required. The protocol then focuses on achieving *completeness*, which ensures that any node $\mathcal{P}_i$ missing its valid share $\mathbf{A}(x, i)$ can successfully recover it. To enable recovery with an amortized linear communication overhead, the process is structured into two rounds. In the first round, each happy node $\mathcal{P}_i$ with valid evaluations $(\mathbf{A}(1, i), E(1, i)), \dots, (\mathbf{A}(n, i), E(n, i))$ forwards each pair $(\mathbf{A}(l, i), E(l, i))$ to each node $\mathcal{P}_l$ (Lines 9-10). Upon receiving $(\mathbf{A}(i, j), E(i, j))$ from node $\mathcal{P}_j$ as node $\mathcal{P}_i$, it verifies the evaluation correctness through the script $v(x, y)$ (Lines 11-12). When collecting correct evaluations from at least $t + 1$ different nodes, node $\mathcal{P}_i$ interpolates them into the column polynomials $\mathbf{A}(i, y)$ (Lines 13-15). In the second round, node $\mathcal{P}_i$ sends to each $\mathcal{P}_k$ the evaluations $\mathbf{A}(i, k)$ on the reconstructed $\mathbf{A}(i, y)$ (Line 16). This allows every node to gather at least $n - t$ evaluations and apply the robust polynomial interpolation algorithm to recover the row polynomial $\mathbf{A}(x, i)$ as node $\mathcal{P}_i$ (Lines 17-19). The recovery process concludes by decoding $\mathbf{A}(x, i)$ into the corresponding valid share $[\mathbf{a}]_i$ (Lines 20-21).

5 GoSSamer-ADPSS

GoSSamer-ADPSS is a batched ADPSS protocol (defined in Section 2.4) that enables the resharing of Shamir shares [65] for a batch of secrets from an old committee to a new committee. Its public input includes the field $\mathbb{Z}_p$ and the batch size b . In the old committee com , the number of nodes is n and the threshold is t , while in the new committee com' , these parameters are n' and t' . Each node $\mathcal{P}_i$ in the old committee privately inputs the (n, t) -threshold share $[\mathbf{s}]_i$ of b secrets $\mathbf{s}$. The protocol outputs the (n', t') -threshold share $[\tilde{\mathbf{s}}]_i$ of the secrets $\mathbf{s}$ to each $\mathcal{P}'_i$ in the new committee, for all $i \in [n']$. Appendix G gives the theoretical analysis of GoSSamer-ADPSS.

In Phase ①, each node in the old committee initiates two lightweight batched ACSS instances, and respectively shares a small batch of random values among the old and new committees. To ensure the shared values are the same across the two committees, a lightweight dual-committee sharing consistency verification protocol is executed. The old committee then runs an MVBA protocol to agree on $n - t$ valid ACSS instances, and informs their indices to the new committee. As a result, the old committee extracts a sharing $[\mathbf{u}]$ and the new committee extracts a corresponding sharing $[\bar{\mathbf{u}}]$ of the same random mask vector $\mathbf{u}$. In Phase ②, each node in the old committee masks its secret share $[\mathbf{s}]$ by locally computing $[\mathbf{s}] + [\mathbf{u}]$. The masked values are then jointly reconstructed through our linear-communication dual-committee reconstruction mechanism, thereby producing the vector of masked secrets $\mathbf{s} + \mathbf{u}$ within the new committee. In Phase ③, each node in the new committee subtracts its own share $[\bar{\mathbf{u}}]$ of the mask from the reconstructed $\mathbf{s} + \mathbf{u}$ to obtain its final share $[\bar{\mathbf{s}}]$ of the secrets $\mathbf{s}$.

5.1 Sub-protocol: Lightweight Sharing Consistency Verification

To eliminate ADPSS's dependence on the commitment-based ACSS schemes, and further enable a fully lightweight ADPSS protocol, we introduce a new component: lightweight dual-committee sharing consistency verification, as presented in Algorithm 3. In this algorithm, we consider two committees, denoted as com and com' respectively, which execute symmetric operations. The procedure in algorithm 3 is illustrated from the perspective of a node in com ; symmetric steps of com' are highlighted with black-bordered boxes.

Problem formulation. Each of the two committees, com and com' , holds the Shamir sharing of b secrets $\mathbf{a}$. The objective is to ensure two *consistent* sharings; namely, reconstruction from either committee yields the same underlying secrets. Meanwhile, no information about the secrets is disclosed.

Definition 1 (Sharing Consistency). Let $\mathbf{a}$ be a vector of b secrets. Suppose committee com of size n holds an (n, t) -sharing of $\mathbf{a}$, i.e., each node $\mathcal{P}_i$ holds a share $[\mathbf{a}]_i = \mathbf{f}(i)$, where $\mathbf{f}(x)$ is a vector of t -degree polynomials, which satisfies $\mathbf{f}(0) = \mathbf{a}$. Similarly, committee com' holds an (n', t') -sharing of $\bar{\mathbf{a}}$. We define the two sharings to be *consistent* if $\mathbf{a} = \bar{\mathbf{a}}$.

Protocol Description. Assume that each node $\mathcal{P}_i$ in committee com holds a secret share $[\mathbf{a}]_i$ of b secrets $\mathbf{a}$, along with a blinding share $[q]_i$. Similarly, each node $\mathcal{P}'_i$ in the other committee com' holds $[\bar{\mathbf{a}}]_i$ and $[\bar{q}]_i$. The protocol begins by letting each committee run a random beacon instance to distributively open a random seed without relying on a trusted party. The seed is then multicast to the other committee (Lines 1-3). After receiving enough seeds from at least one honest node in the other committee, and obtaining the seed of its own committee via random beacon, each node derives a challenge

Algorithm 3 dual-committee sharing consistency verification	
Public Input: field $\mathbb{Z}_p$, the number of nodes and threshold (n, t) in committee com and (n', t') in committee com' , batch size b	
Private Input:	
• as node $\mathcal{P}_i$ in committee com : Shamir share $[\mathbf{a}]_i, [q]_i$	
• as $\text{node } \mathcal{P}'_i$ in committee com' : Shamir share $[\bar{\mathbf{a}}]_i, [\bar{q}]_i$	
1: upon initialization do	► as node $\mathcal{P}_i$ or node $\mathcal{P}'_i$
2: • seed $(\text{seed}) \leftarrow \text{beacon.open}$	
3: • multicast seed (seed) to the other committee	
4: upon receiving enough seed' (seed) from the other committee do	
5: ► enough - at least 1 value comes from an honest node	
6: upon outputting seed (seed) from beacon.open do	
7: • extract the b -dimension challenge $\mathbf{r}$ through $H(\text{seed}, \text{seed}')$	
8: • calculate $[w]_i := \mathbf{r} \cdot [\mathbf{a}]_i + [q]_i$ $[\bar{w}]_i := \mathbf{r} \cdot [\bar{\mathbf{a}}]_i + [\bar{q}]_i$	
9: • multicast $[w]_i$ $[\bar{w}]_i$ among its own committee	
10: upon receiving $[w]_j$ $[\bar{w}]_j$ from node $\mathcal{P}_j$ $\text{node } \mathcal{P}'_j$ do	
11: • run robust interpolation by collecting $(j, [w]_j)$ $(j, [\bar{w}]_j)$	
12: upon outputting $w(x)$ $\bar{w}(x)$ from robust interpolation do	
13: • multicast $w(0)$ $\bar{w}(0)$ to the other committee	
14: upon receiving enough $\bar{w}(0)$ $w(0)$ from the other committee do	
15: if $\bar{w}(0) = w(0)$ do output $[\mathbf{a}]_i$ $[\bar{\mathbf{a}}]_i$	► $w(0) = w, \bar{w}(0) = \bar{w}$

vector $\mathbf{r}$ of length b by hashing the two seeds (Lines 4-7). Then, each node in com calculates a random linear transformation $[w]_i$ from its secret share $[\mathbf{a}]_i$ and blinding share $[q]_i$ through equation $[w]_i := \mathbf{r} \cdot [\mathbf{a}]_i + [q]_i$, and multicasts it within its own committee. All nodes in com' perform the symmetric operations (Lines 8-9).

By collecting $(j, [w]_j)$ sent from different nodes $\mathcal{P}_j$, each node in the committee com can perform a robust polynomial interpolation to reconstruct the corresponding polynomial $w(x)$, and multicast its evaluation at $x = 0$ to the other committee com' . Similarly, nodes in committee com' get $\bar{w}(x)$ from the robust polynomial interpolation and multicast $\bar{w}(0)$ to com (Lines 10-13). Finally, if $w(0) = \bar{w}(0)$, there is $w = \bar{w}$ and thus $\mathbf{a} = \bar{\mathbf{a}}$ except with negligible probability. This confirms that the two committees hold consistent sharings. Consequently, each node in committee com outputs its local share $[\mathbf{a}]_i$, and each node in committee com' outputs $[\bar{\mathbf{a}}]_i$. (Lines 14-15).

Eliminating the cost of random beacon. To ensure that this component is generally applicable to any input Shamir sharing, we employ random beacon to generate the seed, from which the challenge vector is derived. Notably, when applying this component to ADPSS, the cost of the random beacon can be eliminated. Recall that ADPSS utilizes ACSS to generate the Shamir sharings, from which the challenge vector can be derived in a non-interactive approach from the public commitment through Fiat-Shamir transformation. Specifically, in lightweight ACSS like GoSSamer-ACSS, the challenge is derived by hashing the Merkle root output in SMD.

5.2 Concrete Scheme of GoSSamer-ADPSS

Algorithm 4 specifies the procedure of GoSSamer-ADPSS, and a detailed explanation is provided below.

Phase ① : dual-committee random sharing.

The objective of this phase is to generate consistent sharings of b random values for both the old and new committees, without revealing the underlying random values⁴.

Step 1: share. Each node $\mathcal{P}_i$ in the old committee independently samples a batch of $\hat{b} = b/(t+1)$ random values $\mathbf{a}_i$, and then invokes the lightweight dual-committee ACSS (Lines 1-3); the definition of this primitive is provided in Appendix F. In the dual-committee ACSS, node $\mathcal{P}_i$ further samples a blinding value q_i , and initiates two independent, batched ACSS instances to share the secrets $(\mathbf{a}_i, q_i)$ to the old and new committees, respectively (Lines 4-6). To ensure that the sharings held by the two committees correspond to the same underlying secrets, both committees invoke the lightweight dual-committee sharing consistency verification, where each q_i is consumed as a blinding value (Lines 7-8).

Step 2: consensus. In an asynchronous network, ADPSS must proceed once $n-t$ ACSS instances have outputted, because the t instances that the corrupted nodes never initiate are indistinguishable from the t instances that are initiated by honest nodes but arbitrarily delayed by the adversary. To reach consensus on these ACSS instances, each node waits until it outputs from $n-t$ ACSS instances, and then submits the corresponding indices into MVBA (Lines 9-11). All nodes in the old committee wait for the MVBA protocol to produce a common index set Q_{ss} agreed upon by all participating nodes. The old committee then multicasts Q_{ss} to the new committee (Lines 12-14). MVBA employs an external predicate [34, 77] to guarantee that $|Q_{ss}| = n-t$ and that every ACSS instance in Q_{ss} eventually outputs at all honest nodes.

Step 3: extract. As up to t ACSS instances in the set Q_{ss} may be initiated by corrupted dealers, each node locally performs a random extraction step to eliminate their influence while retaining as many useful secrets as possible. Following the technique commonly employed in batched protocols [17, 34, 50], every node deterministically calculates $n-2t$ shares from the $n-t$ shares obtained through the ACSS instances in Q_{ss} , using the hyper-invertible matrix. For ease of description, we assume $\hat{b} = 1$, so that each share $[\mathbf{a}_j]_i$, which is distributed from $\mathcal{P}_j$ to $\mathcal{P}_i$, is a scalar. After collecting $[\mathbf{a}_j]_i$ for all $j \in Q_{ss}$, node $\mathcal{P}_i$ assembles these $n-t$ shares into a vector and multiplies it by a hyper-invertible matrix of size $(n-t) \times (n-2t)$ to obtain the vector $[\mathbf{u}]_i$ of $n-2t$ shares. As a result, when $\hat{b} = b/(t+1)$, the resulting vector is b -dimension (Lines 15-16)

Phase ②: lightweight secret masking.

The objective of this phase is to conceal the sharing $[\mathbf{s}]$ of the secrets by adding the sharing $[\mathbf{u}]$ of the random masks, and then to reconstruct the masked secrets $\mathbf{s} + \mathbf{u}$.

Each node $\mathcal{P}_i$ in the old committee locally adds the random-mask share $[\mathbf{u}]_i$ to its secret share $[\mathbf{s}]_i$, thus obtaining $[\mathbf{s}]_i + [\mathbf{u}]_i$. The vector $[\mathbf{s}]_i + [\mathbf{u}]_i$ is then partitioned into $\hat{b}$ groups, each

⁴This phase requires the completeness property, i.e., all nodes obtain a valid share. In contrast, random beacon imposes no such requirement.

Algorithm 4 GoSSamer-ADPSS

Public Input: field $\mathbb{Z}_p$, batch size b , the committee size n and threshold t in the old committee com , (n', t') in the new committee com'
Private Input: share $[\mathbf{s}]_i$ of b secrets $\mathbf{s}$ as an old committee node $\mathcal{P}_i$
output: share $[\bar{\mathbf{s}}]_i$ of secrets $\mathbf{s}$ for each new committee node $\mathcal{P}'_i$

Phase ①: 2com random sharing (as old committee node $\mathcal{P}_i$)

- 1: **upon** invocation by $\mathcal{P}_i$ in old committee com **do** ▶ Step 1: share
- 2: • sample a vector $\mathbf{a}_i$ of $\hat{b} = b/(t+1)$ random values
- 3: • invoke dual-committee ACSS by:
 - 4: ◦ sample a random blinding value q_i
 - 5: ◦ share $(\mathbf{a}_i, q_i)$ to the old committee through ACSS
 - 6: ◦ share $(\mathbf{a}_i, q_i)$ to the new committee through ACSS
 - 7: ◦ invoke Algorithm 3 for sharing consistency verification
- 8: ▶ verify if the dealer shares the same secrets in both committees
- 9: **upon** outputting $[\mathbf{a}_j]_i$ from the ACSS initiated by node $\mathcal{P}_j$ **do**
- 10: • add index j into local set Q_{ss} ▶ Step 2: consensus
- 11: **upon** $|Q_{ss}| = n-t$ **do** invoke MVBA with input Q_{ss}
- 12: **upon** outputting the common set Q_{ss} in MVBA **do**
- 13: ▶ $|Q_{ss}| = n-t$ and all ACSS instances in Q_{ss} output successfully
- 14: • multicast Q_{ss} to all nodes in the new committee com'
- 15: ▶ Step 3: extract
- 16: • deterministically extract $[\mathbf{u}]_i$ from $[\mathbf{a}_j]_i, j \in Q_{ss}$ through hyper-invertible matrix ▶ extract shares of $\hat{b}(t+1)$ random secrets from $\hat{b}(n-t)$ secrets, where at most $\hat{b}t$ secrets come from corrupted nodes

Phase ②: lightweight secret masking (as old committee node $\mathcal{P}_i$)

- 17: **upon** outputting $[\mathbf{u}]_i$ from last phase **do**
- 18: • divide $[\mathbf{s}]_i + [\mathbf{u}]_i$ into $\hat{b}$ groups of size $(t+1)$
- 19: **for** each group j **do**
 - 20: ◦ interpolate the $(t+1)$ elements into t -degree $F_j(x, i)$
 - 21: ◦ $\mathbf{F}(x, i) := \mathbf{F}(x, i) \parallel F_j(x, i)$
- 22: • send $\mathbf{F}(j, i)$ to each node $\mathcal{P}_j$ for all $j \in [n]$
- 23: **upon** receiving $\mathbf{F}(i, j)$ from $\mathcal{P}_j$ **do**
- 24: • run robust interpolation by collecting $(j, \mathbf{F}(i, j))$
- 25: **upon** outputting $\mathbf{F}(i, y)$ from robust interpolation **do**
- 26: • multicast $\mathbf{F}(i, 0)$ to all nodes in new committee com'

Phase ③: secret share derivation (as new committee node $\mathcal{P}'_i$)

- 27: **upon** receiving Q_{ss} from the old committee for $(t+1)$ times **do**
- 28: **upon** outputting $[\bar{\mathbf{a}}_j]_i$ for all $j \in Q_{ss}$ from ACSS **do**
- 29: • extract $[\bar{\mathbf{u}}]_i$ from $[\bar{\mathbf{a}}_j]_i, j \in Q_{ss}$ ▶ same as Lines 15-16
- 30: **upon** receiving $\mathbf{F}(j, 0)$ from $\mathcal{P}_j$ in the old committee **do**
- 31: • run robust interpolation by collecting $(j, \mathbf{F}(j, 0))$
- 32: **upon** outputting $\mathbf{F}(x, 0)$ from robust interpolation **do**
- 33: • decode $\mathbf{F}(x, 0)$ into $\mathbf{s} + \mathbf{u}$ and output $[\bar{\mathbf{s}}]_i := \mathbf{s} + \mathbf{u} - [\bar{\mathbf{u}}]_i$

of size $t+1$, and each group is interpolated into a t -degree polynomial. The resulting $\hat{b}$ polynomials are denoted as $\mathbf{F}(x, i)$, where $\mathbf{F}(x, 0)$ essentially encodes the masked secrets $\mathbf{s} + \mathbf{u}$ (Lines 17-21). Subsequently, each node $\mathcal{P}_i$ sends the evaluation $\mathbf{F}(j, i)$ to each node $\mathcal{P}_j$ within its old committee (Line 22). Upon receiving evaluations from distinct senders, each node $\mathcal{P}_i$ locally runs a robust polynomial interpolation algorithm to reconstruct $\mathbf{F}(i, y)$ (Lines 23-25). Finally, to enable the new committee to recover $\mathbf{F}(x, 0)$ and hence derive the masked secrets $\mathbf{s} + \mathbf{u}$, each node in the old committee multicasts the evaluation $\mathbf{F}(i, 0)$ to all nodes in the new committee (Line 26).

Phase ③: secret share derivation.

The objective of this phase is to derive the new sharing $[\bar{\mathbf{s}}]$ of the secrets $\mathbf{s}$ for all nodes in the new committee.

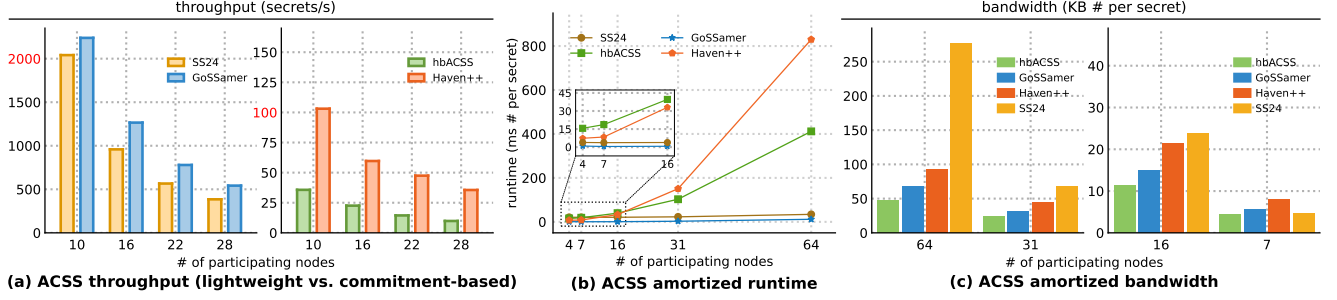


Figure 4: The evaluation results of batched ACSS

Each node $\mathcal{P}_i^t$ in the new committee first extracts its share $[\bar{\mathbf{u}}]_i$ of the b random masks (Lines 27-29). The extraction procedure is identical to that described in Lines 15-16. Subsequently, each node $\mathcal{P}_i^t$ in the new committee collects the evaluation $\mathbf{F}(j, 0)$ sent from distinct old committee nodes $\mathcal{P}_j$, and locally runs robust polynomial interpolation to reconstruct $\mathbf{F}(x, 0)$ (Lines 30-32). From $\mathbf{F}(x, 0)$, the masked secrets $\mathbf{s} + \mathbf{u}$ are decoded, and each node $\mathcal{P}_i^t$ computes its target share $[\bar{\mathbf{s}}]_i$ by subtracting the share of random masks from the masked secrets, i.e., $[\bar{\mathbf{s}}]_i = \mathbf{s} + \mathbf{u} - [\bar{\mathbf{u}}]_i$ (Line 33). It is straightforward to verify that $[\bar{\mathbf{u}}]_i$ is the i -th share of $\mathbf{u}$, thus the new secret sharing $[\bar{\mathbf{s}}]$ is consistent with the original secret sharing $[\mathbf{s}]$.

6 Evaluation

This section evaluates GoSSamer and its deployment in distributed key management. We implement GoSSamer-ACSS, GoSSamer-ADPSS, SS24 [67], and GoSSamer-ACSS-based ADKG that builds upon GS24 [50]. We also reproduce LongLive [77], hbACSS [76], Haven++ [4], and DXK+23 [34]. Our code is built on top of implementations including [4, 34, 36, 76, 77].

Implementation details. The implementations are primarily developed in Python 3.7.13, with the core cryptographic operations delegated to a Rust-based library from Zcash [49]. The BLS12-381 elliptic curve is utilized to support downstream pairing-based primitives. To ensure a fair comparison while minimizing changes to the original implementations, we equip every evaluated implementation with the same building blocks: the reliable broadcast from [35], the MVBA from [36], and the robust interpolation from [43]. Moreover, we implement the secure message distribution protocol from [67], with modifications to support batch processing.

Evaluation environment. Experiments were conducted on the machine with a 16-core Intel i9-12900KF CPU and 64GB of RAM. We utilize a Bash script to simulate a distributed environment consisting of n nodes. Each node is instantiated as an independent process bound to a unique network port, enabling inter-process message exchanges. We adopt a locally simulated environment, following the approaches in LongLive [77], hbACSS [76] and Haven++ [4]. In batched pro-

ocols, the primary performance bottleneck is computational overhead, rather than bandwidth or latency. Nevertheless, to better approximate real-world conditions, we serialize messages and use Asyncio queues to emulate inter-process I/O.

6.1 Evaluation of GoSSamer-ACSS

Baselines. We compare GoSSamer-ACSS against three batched ACSS protocols, including SS24 [67], hbACSS [76], and Haven++ [4]. We first implement the theoretical solution SS24 [67] to facilitate direct comparison with lightweight scheme. For protocols that achieve linear per-secret communication, we choose two representatives hbACSS [76] and Haven++ [4]. Among them, the univariate-polynomial-based scheme hbACSS [76] requires a trusted setup, whereas the bivariate-polynomial-based scheme Haven++ [4] operates without one. Our evaluation focuses on adversarial scenarios where the dealer is Byzantine and may distribute faulty shares to t nodes, which is critical to applications like ADPSS, MPC, ADKG, where t nodes may act as malicious ACSS dealers.

Throughput comparison. Figure 4(a) reports the *throughput* comparison, defined as the number of secrets shared per second. When varying the number of the participating nodes at $n = 10, 16, 22, 28$, lightweight schemes (Figure 4(a), left) share approximately 10^3 secrets/s, whereas the commitment-based protocols (Figure 4(a), right) share 10 - 10^2 secrets/s. Within the lightweight category, GoSSamer-ACSS consistently outperforms SS24 [67], sharing 111–432 additional secrets/s across the evaluated configurations.

Amortized runtime comparison. Figure 4(b) presents the *amortized runtime*, defined as the per-secret runtime. To capture the asymptotic performance of each scheme, we set the number of nodes to $n = 4, 7, 16, 31, 64$ and set the batch size to $n^2 \log n$. The results illustrate that the runtime in GoSSamer-ACSS is only 1.3-11.5% of the runtime in hbACSS [76] and Haven++ [4]. Among lightweight schemes, GoSSamer-ACSS outperforms SS24 [67] by 2.6-27.3%.

Amortized bandwidth comparison. Figure 4(c) reports the *amortized bandwidth usage*, defined as the total volume of data sent by all nodes, divided by the batch size. Both the number of nodes and the batch sizes are configured identically to Fig-

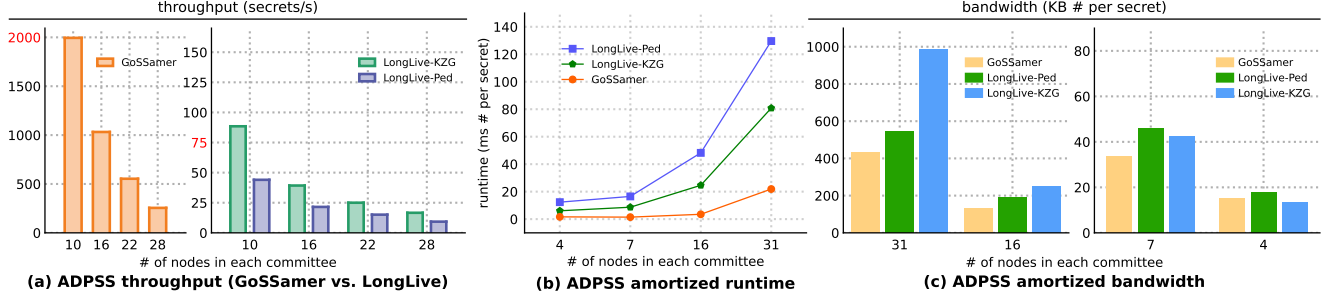


Figure 5: The evaluation results of batched ACSS and batched ADPSS

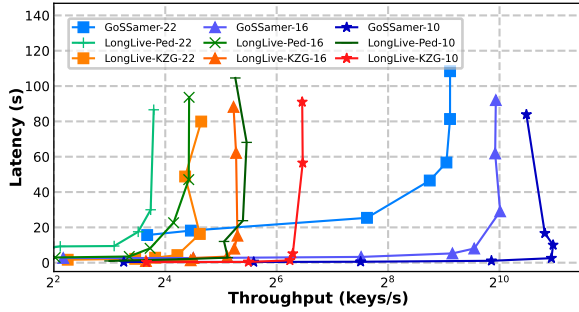


Figure 6: The throughput-latency trade-off in key resharing.

ure 4(b). The results indicate that Haven++ [4], hbACSS [76], and GoSSamer-ACSS achieve linear per-secret communication overhead with respect to n , whereas SS24 incurs a quadratic cost. Relative to hbACSS [76], GoSSamer-ACSS requires slightly more bandwidth (15.1 KB vs. 11.5 KB at $n = 16$). However, our solution is computationally lightweight and removes the trusted setup of hbACSS [76]. Concretely, compared with GoSSamer-ACSS, hbACSS [76] reduces bandwidth by 30.8% but prolongs runtime by 369.7% at $n = 31$.

6.2 Evaluation of GoSSamer-ADPSS

Baseline. We adopt LongLive [77] as our primary baseline. The original LongLive implementation uses the KZG commitment [57] with a trusted setup; we denote this version as LongLive-KZG. We also re-implement the LongLive protocol [77] with the Pedersen commitment to eliminate the trusted setup; we refer to this variant as LongLive-Ped.

Throughput comparison. Figure 5(a) reports the *throughput* comparison. When varying the number of nodes at $n = 10, 16, 22, 28$, GoSSamer (Figure 5(a), left) achieves 14-26 $\times$ more throughput compared to LongLive-KZG, and 27-48 $\times$ compared to LongLive-Ped (Figure 5(a), right).

Amortized runtime comparison. Figure 5(b) presents the *amortized runtime*, measured with the number of nodes in each committee set to $n = 4, 7, 16, 31$ and the batch size set to $n^2 \log n$. The results demonstrate that GoSSamer-ADPSS

Table 3: Runtime (in seconds) in key management lifecycle

n	Key Generation		Key Resharing		Key Retrieval [43]
	DXK+23 [34]	GoSSamer-ACSS-based [50]	LongLive [77]	GoSSamer-ADPSS	
10	650.62	88.76	115.18	8.77	3.35
16	1682.57	151.21	259.05	20.98	5.85
22	3238.00	316.64	397.68	45.37	7.74

achieves significantly lower runtime than other implementations, with its overhead ranging from 14.2-27.1% of LongLive-KZG, and 7.3-13.3% of LongLive-Ped.

Amortized bandwidth comparison Figure 5(c) reports the *amortized bandwidth usage*. Both the number of nodes and the batch sizes are configured identically to Figure 5(b). The results illustrate that GoSSamer-ADPSS incurs relatively low communication cost, followed by LongLive-Ped and LongLive-KZG. Specifically, the bandwidth in GoSSamer is 44.2% of LongLive-KZG and 80.0% of LongLive-Ped at $n = 31$.

6.3 Key Management Evaluation in Web3Auth

Web3Auth [62] is deployed atop the Torus Network, whose mainnet currently consists of around 10 nodes, and will be expanded to roughly 22 nodes. We therefore conduct experiments with $n = 10, 16, 22$.

The runtime in distributed key management lifecycle. We evaluate the runtime of key generation, key resharing, and key retrieval phases in distributed key management. Table 3 reports our evaluation results, where the batch size is set to 10^4 . For key generation, we implement a GoSSamer-ACSS-based ADKG scheme that builds upon GS24 [50], the evaluation results demonstrate that it achieves 7-11 $\times$ faster than the ADKG implementation DXK+23 [34], which we also modified to support batch processing. For key resharing, GoSSamer-ADPSS demonstrates 9-13 $\times$ faster than LongLive [77], markedly shortening the service downtime required for reconfiguration.

The throughput-latency trade-off in key resharing. We further evaluate the batch-processing efficiency in key resharing, with the three batched ADPSS implementations, GoSSamer,

LongLive-Ped [77] and LongLive-KZG [77]. Figure 6 reports our evaluation results in *throughput-latency trade-off*, where throughput is defined as the number of keys reshared per second and latency is the end-to-end runtime. The results demonstrate that lightweight protocol achieves strong performance under batched operation. Specifically, GoSSamer-ADPSS achieves a peak throughput of 2.0×10^3 keys/s with a low latency of 10.03s at $n = 10$. Compared with both LongLive variants, our solution reshapes 1900 additional keys per second at $n = 10$, and 500 additional keys per second at $n = 22$.

7 Applications

Other Applications of GoSSamer-ACSS. GoSSamer-ACSS presents an efficient alternative to existing batched ACSS solutions, featuring lightweight computation, linear per-secret communication, and high batch-processing efficiency. We demonstrate some of its applications in Table 4, including ADKG, random beacon, and asynchronous multi-party computation (AMPC), with more information given in Appendix D.

Other Applications of GoSSamer-ADPSS. (1) *Dynamic MPC service.* MPC enables a committee of servers to evaluate user-defined functions over private inputs [60]. In asynchronous dynamic MPC services [75], ADPSS is utilized to facilitate the secure migration of secret shares under dynamic committee reconfiguration, ensuring that the MPC state persists without restarting or secret leakage. Furthermore, proactive resharing allows costly preprocessing to be performed opportunistically during periods of low network activity and securely retained for future use, particularly in settings such as the BMR Escape Hatch [11, 77]. To support these services efficiently, GoSSamer provides a lightweight solution with optimal per-secret communication. (2) *General share migration service.* In long-lived threshold systems under dynamic committee membership or mobile adversaries, proactive secret resharing is essential for maintaining consistent system status across epochs. For example, Extractable witness encryption on blockchains [47] enables the distributed storage and retrieval of secrets among a committee, where secrets are reshared upon epoch transitions. Dynamic Byzantine fault tolerance protocols [44, 56] and dynamic threshold signatures [30] utilize share resharing for committee reconfiguration in blockchain. Some decentralized identity protocols [59, 74] distributes identity secrets as Shamir shares to enable privacy-preserving identity management. GoSSamer-ADPSS can be utilized as an efficient and readily deployable primitive in these systems.

8 Discussion

MVBA for lightweight protocol. To ensure that GoSSamer-ADPSS operates solely with lightweight cryptographic primitives, the embedded MVBA protocol should also adopt a lightweight design without trusted setup. DDL+24 [32] is one

Table 4: Applications demonstration of GoSSamer-ACSS

Scheme	Lightweight	Communication (# amortized)	Assumption	Reference
Random Beacon	✗	$O(\kappa n^2)$	q -SDH	[1]
	✓	$O(\kappa n^3 \log n)$	RO	[9]
	✓	$O(\kappa n^2)$	RO	GoSSamer-based
ADKG	✗(ACSS)*	$O(\kappa n^2 \log n)$	DL+RO	[7]
	✓(ACSS)*	$O(\kappa n^3)$	DL+RO	[50]
	✓(ACSS)*	$O(\kappa n^2)$	DL+RO	GoSSamer-based
AMPC	✓	$O(\kappa n^4)$	None	[29]
	✓ ^b	$O(\kappa n)$	None	[48]
	✓	$O(\kappa n)$	RO	GoSSamer-based

* We mark the ADKG protocols that can derive public keys from a lightweight ACSS with ✓ and those that rely on a commitment-based ACSS with ✗.

^b Although [48] computes solely over finite-field arithmetic, we use ✓ to indicate that it is not fully lightweight, as it incurs an impractical communication of $O(\kappa n + \kappa^2 n^{12})$.

such candidate, which has a communication complexity of $O(\kappa n^3)$. Nonetheless, MVBA protocols equipped with a common coin setup typically achieve a lower expected runtime, exemplified by the lightweight design Fin [39]. In GoSSamer-ADPSS, establishing the common coin is straightforward because MVBA is executed by the old committee, which has been active for an epoch before initiating the ADPSS protocol, thereby providing sufficient time for coin generation.

Extending GoSSamer-ADPSS for high-threshold reconstruction. The ADPSS protocol LongLive [77] supports a higher reconstruction threshold of $l < 2/3n$ with a higher communication of $O(\kappa n^2)$, when instantiated with the Pedersen commitment. This observation leads to a question of whether our lightweight resharing technique can be extended to accommodate such high-threshold reconstruction. Although this is not our focus, we nevertheless outline a solution. As the threshold $t = n/3$ already constitutes the upper bound for robust interpolation, any reconstruction at a higher threshold necessitates that each secret share $[s]$ be accompanied by an explicit correctness proof. Our solution starts from the lightweight shares generated by GoSSamer-ADPSS with the low reconstruction threshold t . Any node that possesses $l + 1$ such shares, distributively combines them into an evaluation on a l -degree polynomial, following the polynomial-sampling method of DXK+23 [34]. The resulting evaluation yields a high-threshold share $[s]$. To equip the new share with a correctness proof, we adopt GS24’s [50] approach and distributively derive the proof $g^{[s]}$, where g is a group generator. While we leave a fully lightweight construction to future work, the present approach already closes a practical gap: existing high-threshold ADPSS constructions [77] spend roughly 90% of their running time in the underlying commitment-based ACSS; in contrast, our solution derives high-threshold shares and proofs through lightweight ACSS, thereby having the potential of providing concrete efficiency gain.

References

- [1] Ittai Abraham, Philipp Jovanovic, Mary Maller, Sarah Meiklejohn, and Gilad Stern. Bingo: Adaptivity and asynchrony in verifiable secret sharing and distributed key generation. In *Annual International Cryptology Conference*, pages 39–70. Springer, 2023.
- [2] Ittai Abraham, Philipp Jovanovic, Mary Maller, Sarah Meiklejohn, Gilad Stern, and Alin Tomescu. Reaching consensus for asynchronous distributed key generation. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, pages 363–373, 2021.
- [3] Andreea B Alexandru, Erica Blum, Jonathan Katz, and Julian Loss. State machine replication under changing network conditions. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 681–710. Springer, 2022.
- [4] Nicolas Alhaddad, Mayank Varia, and Ziling Yang. Haven++: Batched and packed dual-threshold asynchronous complete secret sharing with applications. *IACR Communications in Cryptology*, 1(4), 2025.
- [5] Nicolas Alhaddad, Mayank Varia, and Haibin Zhang. High-threshold avss with optimal communication complexity. In *International Conference on Financial Cryptography and Data Security*, pages 479–498. Springer, 2021.
- [6] Lit Association. Lit protocol. <https://www.litprotocol.com>.
- [7] Renas Bacho, Julian Loss, Gilad Stern, and Benedikt Wagner. Harts: High-threshold, adaptively secure, and robust threshold schnorr signatures. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 104–140. Springer, 2024.
- [8] Michael Backes, Amit Datta, and Aniket Kate. Asynchronous computational vss with reduced communication complexity. In *Cryptographers’ Track at the RSA Conference*, pages 259–276. Springer, 2013.
- [9] Akhil Bandrupalli, Adithya Bhat, Saurabh Bagchi, Aniket Kate, and Michael K Reiter. Random beacons in monte carlo: Efficient asynchronous random beacon without threshold cryptography. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 2621–2635, 2024.
- [10] Joshua Baron, Karim El Defrawy, Joshua Lampkins, and Rafail Ostrovsky. Communication-optimal proactive secret sharing for dynamic groups. In *International conference on applied cryptography and network security*, pages 23–41. Springer, 2015.
- [11] Donald Beaver, Silvio Micali, and Phillip Rogaway. The round complexity of secure protocols. In *Proceedings of the twenty-second annual ACM symposium on Theory of computing*, pages 503–513, 1990.
- [12] Zuzana Beerliová-Trubíniová and Martin Hirt. Simple and efficient perfectly-secure asynchronous mpc. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 376–392. Springer, 2007.
- [13] Michael Ben-Or, Ran Canetti, and Oded Goldreich. Asynchronous secure computation. In *Proceedings of the twenty-fifth annual ACM symposium on Theory of computing*, pages 52–61, 1993.
- [14] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast reed-solomon interactive oracle proofs of proximity. In *45th international colloquium on automata, languages, and programming (icalp 2018)*, pages 14–1. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2018.
- [15] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf. Deep-fri: sampling outside the box improves soundness. *arXiv preprint arXiv:1903.12243*, 2019.
- [16] Fabrice Benhamouda, Craig Gentry, Sergey Gorbunov, Shai Halevi, Hugo Krawczyk, Chengyu Lin, Tal Rabin, and Leonid Reyzin. Can a public blockchain keep a secret? In *Theory of Cryptography Conference*, pages 260–290. Springer, 2020.
- [17] Fabrice Benhamouda, Shai Halevi, Hugo Krawczyk, Yiping Ma, and Tal Rabin. Sprint: High-throughput robust distributed schnorr signatures. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 62–91. Springer, 2024.
- [18] Adithya Bhat, Nibesh Shrestha, Aniket Kate, and Kartik Nayak. Optrand: Optimistically responsive reconfigurable distributed randomness. In *Network and Distributed System Security Symposium (NDSS)*, 2023.
- [19] Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. Random oracles in a quantum world. In *International conference on the theory and application of cryptology and information security*, pages 41–69. Springer, 2011.
- [20] Jonathan Bootle, Andrea Cerulli, Pyrros Chaidos, Jens Groth, and Christophe Petit. Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 327–357. Springer, 2016.

- [21] Gabriel Bracha. Asynchronous byzantine agreement protocols. *Information and Computation*, 75(2):130–143, 1987.
- [22] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE symposium on security and privacy (SP)*, pages 315–334. IEEE, 2018.
- [23] Christian Cachin, Klaus Kursawe, Anna Lysyanskaya, and Reto Stroh. Asynchronous verifiable secret sharing and proactive cryptosystems. In *Proceedings of the 9th ACM Conference on Computer and Communications Security*, pages 88–97, 2002.
- [24] Christian Cachin, Klaus Kursawe, Frank Petzold, and Victor Shoup. Secure and efficient asynchronous broadcast protocols. In *Annual International Cryptology Conference*, pages 524–541. Springer, 2001.
- [25] Christian Cachin and Stefano Tessaro. Asynchronous verifiable information dispersal. In *24th IEEE Symposium on Reliable Distributed Systems (SRDS’05)*, pages 191–201. IEEE, 2005.
- [26] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*, pages 136–145. IEEE, 2001.
- [27] Ashish Choudhury, Martin Hirt, and Arpita Patra. Asynchronous multiparty computation with linear communication complexity. In *International Symposium on Distributed Computing*, pages 388–402. Springer, 2013.
- [28] Ashish Choudhury and Arpita Patra. An efficient framework for unconditionally secure multiparty computation. *IEEE Transactions on Information Theory*, 63(1):428–468, 2016.
- [29] Ashish Choudhury and Arpita Patra. On the communication efficiency of statistically secure asynchronous mpc with optimal resilience. *Journal of Cryptology*, 36(2):13, 2023.
- [30] Annalisa Cimatti, Francesco De Sclavis, Giuseppe Galano, Sara Giammusso, Michela Iezzi, Antonio Muci, Matteo Nardelli, and Marco Pedicini. Dynamic-frost: Schnorr threshold signatures with a flexible committee. *Journal of Mathematical Cryptology*, 19(1):20240045, 2025.
- [31] Ivan Damgård, Martin Geisler, Mikkel Krøigaard, and Jesper Buus Nielsen. Asynchronous multiparty computation: Theory and implementation. In *International workshop on public key cryptography*, pages 160–179. Springer, 2009.
- [32] Sourav Das, Sisi Duan, Shengqi Liu, Atsuki Momose, Ling Ren, and Victor Shoup. Asynchronous consensus without trusted setup or public-key cryptography. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 3242–3256, 2024.
- [33] Sourav Das and Ling Ren. Adaptively secure bls threshold signatures from ddh and co-cdh. In *Annual International Cryptology Conference*, pages 251–284. Springer, 2024.
- [34] Sourav Das, Zhuolun Xiang, Lefteris Kokoris-Kogias, and Ling Ren. Practical asynchronous high-threshold distributed key generation and distributed polynomial sampling. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 5359–5376, 2023.
- [35] Sourav Das, Zhuolun Xiang, and Ling Ren. Asynchronous data dissemination and its applications. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2705–2721, 2021.
- [36] Sourav Das, Thomas Yurek, Zhuolun Xiang, Andrew Miller, Lefteris Kokoris-Kogias, and Ling Ren. Practical asynchronous distributed key generation. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 2518–2534. IEEE, 2022.
- [37] Yvo Desmedt and Sushil Jajodia. Redistributing secret shares to new access structures and its applications. Technical report, Technical Report ISSE TR-97-01, George Mason University, 1997.
- [38] Yvo G Desmedt. Threshold cryptography. *European Transactions on Telecommunications*, 5(4):449–458, 1994.
- [39] Sisi Duan, Xin Wang, and Haibin Zhang. Fin: Practical signature-free asynchronous common subset in constant time. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 815–829, 2023.
- [40] Cynthia Dwork, Krishnaram Kenthapadi, Frank McSherry, Ilya Mironov, and Moni Naor. Our data, ourselves: Privacy via distributed noise generation. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 486–503. Springer, 2006.
- [41] Hanwen Feng, Yingzi Gao, Yuan Lu, Qiang Tang, and Jing Xu. Practical asynchronous distributed key reconfiguration and its applications. *Cryptology ePrint Archive*, 2025.

- [42] Hanwen Feng and Qiang Tang. Asymptotically optimal adaptive asynchronous common coin and dkg with silent setup. *Cryptology ePrint Archive*, 2024.
- [43] Shuhong Gao. A new algorithm for decoding reed-solomon codes. In *Communications, information and network security*, pages 55–68. Springer, 2003.
- [44] Yingzi Gao, Yuan Lu, Zhenliang Lu, Qiang Tang, Yuyi Wang, and Jing Xu. Turritopsis: Practical dynamic asynchronous bft. *IEEE Transactions on Information Forensics and Security*, 2025.
- [45] Yingzi Gao, Yuan Lu, Zhenliang Lu, Qiang Tang, Jing Xu, and Zhenfeng Zhang. Dumbo-ng: Fast asynchronous bft consensus with throughput-oblivious latency. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 1187–1201, 2022.
- [46] Rosario Gennaro, Stanisław Jarecki, Hugo Krawczyk, and Tal Rabin. Secure distributed key generation for discrete-log based cryptosystems. In *Advances in Cryptology—EUROCRYPT’99: International Conference on the Theory and Application of Cryptographic Techniques Prague, Czech Republic, May 2–6, 1999 Proceedings 18*, pages 295–310. Springer, 1999.
- [47] Vipul Goyal, Abhiram Kothapalli, Elisaweta Masserova, Bryan Parno, and Yifan Song. Storing and retrieving secrets on a blockchain. In *IACR International Conference on Public-Key Cryptography*, pages 252–282. Springer, 2022.
- [48] Vipul Goyal, Chen-Da Liu-Zhang, and Yifan Song. Towards achieving asynchronous mpc with linear communication and optimal resilience. In *Annual International Cryptology Conference*, pages 170–206. Springer, 2024.
- [49] Jack Grigg and Sean Bowe. zkcrypto/pairing. <https://github.com/zkcrypto/pairing>.
- [50] Jens Groth and Victor Shoup. Fast batched asynchronous distributed key generation. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 370–400. Springer, 2024.
- [51] Amir Herzberg, Stanisław Jarecki, Hugo Krawczyk, and Moti Yung. Proactive secret sharing or: How to cope with perpetual leakage. In *annual international cryptology conference*, pages 339–352. Springer, 1995.
- [52] Martin Hirt and Ueli Maurer. Robustness for free in unconditional multi-party computation. In *Annual International Cryptology Conference*, pages 101–118. Springer, 2001.
- [53] Bin Hu, Jianwei Liu, Zhenliang Lu, Qiang Tang, Zhuolun Xiang, and Zongyang Zhang. Optimistic asynchronous dynamic-committee proactive secret sharing. *Cryptology ePrint Archive*, 2025.
- [54] Bin Hu, Zongyang Zhang, Han Chen, You Zhou, Huazu Jiang, and Jianwei Liu. Dycaps: Asynchronous proactive secret sharing for dynamic committees. *IACR Cryptol. ePrint Arch.*, 2022(1169), 2022.
- [55] Xiaoyu Ji, Junru Li, and Yifan Song. Linear-communication asynchronous complete secret sharing with optimal resilience. In *Annual International Cryptology Conference*, pages 418–453. Springer, 2024.
- [56] Aniket Kate, Pratyay Mukherjee, Samipa Samanta, and Pratik Sarkar. Dyna-hints: Silent threshold signatures for dynamic committees. *Cryptology ePrint Archive*, 2025.
- [57] Aniket Kate, Gregory M Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *International conference on the theory and application of cryptology and information security*, pages 177–194. Springer, 2010.
- [58] Yizhong Liu, Xinxin Xing, Haosu Cheng, Dawei Li, Zhenyu Guan, Jianwei Liu, and Qianhong Wu. A flexible sharding blockchain protocol based on cross-shard byzantine fault tolerance. *IEEE Transactions on Information Forensics and Security*, 18:2276–2291, 2023.
- [59] Yizhong Liu, Zedan Zhao, Boyu Zhao, Feiang Ran, Xun Lin, Dawei Li, and Zhenyu Guan. Fully anonymous decentralized identity supporting threshold traceability with practical blockchain. In *Proceedings of the ACM on Web Conference 2025*, pages 3628–3638, 2025.
- [60] Donghang Lu, Thomas Yurek, Samarth Kulshreshtha, Rahul Govind, Aniket Kate, and Andrew Miller. Honeybadgermpc and asynchromix: Practical asynchronous mpc and its application to anonymous communication. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 887–903, 2019.
- [61] Sai Krishna Deepak Maram, Fan Zhang, Lun Wang, Andrew Low, Yupeng Zhang, Ari Juels, and Dawn Song. Churp: dynamic-committee proactive secret sharing. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 2369–2386, 2019.
- [62] MetaMask. Web3auth. <https://web3auth.io/docs/infrastructure/nodes-and-dkg>.

- [63] Arpita Patra, Ashish Choudhury, and C Pandu Rangan. Efficient asynchronous verifiable secret sharing and multiparty computation. *Journal of Cryptology*, 28:49–109, 2015.
- [64] David A Schultz, Barbara Liskov, and Moses Liskov. Mobile proactive secret sharing. In *Proceedings of the twenty-seventh ACM symposium on Principles of distributed computing*, pages 458–458, 2008.
- [65] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
- [66] Adi Shamir. How to share a secret (1979). 2021.
- [67] Victor Shoup and Nigel P Smart. Lightweight asynchronous verifiable secret sharing with optimal resilience. *Journal of Cryptology*, 37(3):27, 2024.
- [68] K Srinathan and C Pandu Rangan. Efficient asynchronous secure multiparty distributed computation. In *International Conference on Cryptology in India*, pages 117–129. Springer, 2000.
- [69] Alin Tomescu, Robert Chen, Yiming Zheng, Ittai Abraham, Benny Pinkas, Guy Golan Gueta, and Srinivas Devadas. Towards scalable threshold cryptosystems. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 877–893. IEEE, 2020.
- [70] Robin Vassantlal, Eduardo Alchieri, Bernardo Ferreira, and Alysson Bessani. Cobra: Dynamic proactive secret sharing for confidential bft services. In *2022 IEEE symposium on security and privacy (SP)*, pages 1335–1353. IEEE, 2022.
- [71] L Welch and R Scholtz. Continued fractions and berlekamp’s algorithm. *IEEE Transactions on Information Theory*, 25(1):19–27, 2003.
- [72] Theodore M Wong, Chenxi Wang, and Jeannette M Wing. Verifiable secret redistribution for archive systems. In *First International IEEE Security in Storage Workshop, 2002. Proceedings.*, pages 94–105. IEEE, 2002.
- [73] Yunzhou Yan, Yu Xia, and Srinivas Devadas. Shanrang: Fully asynchronous proactive secret sharing with dynamic committees. *Cryptology ePrint Archive*, 2022.
- [74] Jie Yin, Yang Xiao, Qian Chen, Yongzhi Lim, Xuefeng Liu, Qingqi Pei, and Jianying Zhou. Dp-did: A dynamic and proactive decentralized identity system. *IEEE Transactions on Information Forensics and Security*, 2025.
- [75] Wenxuan Yu, Minghui Xu, Bing Wu, Sisi Duan, and Xiuzhen Cheng. Ad-mpc: Fully asynchronous dynamic mpc with guaranteed output delivery. In *Proceedings of the nineteenth annual ACM symposium on Principles of distributed computing*, 2025.
- [76] Thomas Yurek, Licheng Luo, Jaiden Fairoze, Aniket Kate, and Andrew Miller. hbacss: How to robustly share many secrets. In *Proceedings of the Network and Distributed System Security Symposium (NDSS) 2022*, 2022.
- [77] Thomas Yurek, Zhuolun Xiang, Yu Xia, and Andrew Miller. Long live the honey badger: Robust asynchronous {DPSS} and its applications. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 5413–5430, 2023.
- [78] Lidong Zhou, Fred B Schneider, and Robbert Van Renesse. Apss: Proactive secret sharing in asynchronous systems. *ACM transactions on information and system security (TISSEC)*, 8(3):259–286, 2005.

A Supplementary for Related Work

A.1 Related Work of ACSS

ACSS is a basic tool in building asynchronous multi-party computation and asynchronous threshold cryptographic systems, such as asynchronous distributed key generation and asynchronous dynamic proactive secret sharing. In the following, we review the related work in the most relevant settings.

Information-theoretic ACSS. Prior work in this area targets either perfect security or statistical security. Although perfect security offers the strongest guarantees, perfectly secure protocols [12, 27, 28, 63, 68] withstand at most $1/4$ corrupted nodes [13], while the optimal resilience in the asynchronous network is $1/3$. Under statistical security, CP23 [29] achieves a per-secret communication cost of $O(\kappa n^3)$. JLS24 [55] reduces this cost to the lower bound of $O(\kappa n)$, but its total communication complexity reaches $O(\kappa b n + \kappa^2 n^{12})$.

Computational secure ACSS. CKA+02 [23], BDK13 [8] and DXR21 [35] study asynchronous verifiable secret sharing without the completeness guarantee. CKA+02 [23] incurs a communication complexity of $O(\kappa n^3)$. Leveraging KZG commitment [57], BDK13 [8] reduces this cost to $O(\kappa n^2)$. DXR21 [35] attains the same $O(\kappa n^2)$ complexity without assuming a trusted setup. Haven [5] presents a high-threshold ACSS protocol that incurs $O(\kappa n^2)$ communication when instantiated with KZG commitment [57], and $O(\kappa n^2 \log n)$ when instantiated with setup-free commitment schemes [14, 15, 20, 22]. The term “high-threshold” indicates that the protocol supports any reconstruction threshold in the range of $[f + 1, n - f]$, where $f < n/3$ denotes the maximum number of Byzantine faults the system can tolerate. hbACSS [76] provides a series of batched ACSS protocols, where the variant that employs KZG commitments [57] achieves a per-secret communication complexity of $O(\kappa n)$. Bingo [1] and Haven++ [4] attain the same $O(\kappa n)$ per-secret cost for high-threshold ACSS by leveraging bivariate polynomials. The two schemes diverge in their setup requirements: Bingo [1] depends on the KZG commitment [57], whereas Haven++ [4] eliminates this trusted setup with the Pedersen commitment. The above schemes rely on additively homomorphic polynomial commitments for share verification; the resulting computational overhead constrains their batch-processing efficiency. SS24 [67] presents the first lightweight batched ACSS protocol, enhancing batch-processing efficiency through only lightweight cryptographic primitives like hash functions. Nevertheless, the communication complexity of SS24 [67] reaches $O(\kappa n^2)$ per secret when the dealer misbehaves.

Our solution, GoSSamer-ACSS, is a computationally secure batched ACSS protocol that simultaneously delivers lightweight computation, linear per-secret communication, and high concrete efficiency.

A.2 Related Work of ADPSS

Numerous works have studied the problem of proactive share refreshing under various network assumptions [10, 16, 23, 37, 47, 51, 53, 54, 61, 64, 70, 72, 73, 77, 78]. First, we review the proactive secret sharing protocols (PSS) and dynamic proactive secret sharing protocols (DPSS) under the synchronous and partially synchronous networks.

Synchronous protocols. HJK+95 [51] initiates the study of proactive secret sharing, which defends against the gradual corruption of mobile adversaries through periodically share refreshing with a communication complexity of $O(\kappa n^3)$. DJ97 [37] proposes the first dynamic proactive secret sharing against passive adversaries. WWW02 [72] improves upon DJ97 [37] by providing verifiability with an exponential communication complexity. BDL+15 [10] introduces a virtualization-based approach to attain constant communication, which operates under a statistical security model. However, the protocol requires the packed usage of secrets and an impractically large committee size. CHURP [61] dynamically refreshes the sharing through bivariate-polynomial-based secret sharing of zero, which achieves a communication complexity of $O(\kappa n^2)$ in the best case and $O(\kappa n^3)$ in the worst case. GKM+22 [47] utilizes the randomness extraction technique for the batch processing, which achieves a per-secret communication complexity of $O(\kappa n^2)$ in the worst-case bound.

Partially synchronous protocols. Schultz-MPSS [64] presents a protocol for dynamic proactive resharing in partially synchronous networks. It employs a primary-based approach with a view-change mechanism, incurring a communication complexity of $O(\kappa n^4)$ while guaranteeing liveness during periods of synchrony. Improving upon this, COBRA [70] reduces the communication complexity to $O(\kappa n^3)$ in the optimistic case, through a distributed polynomial generation mechanism coupled with a recovery mechanism.

Our study focuses on the asynchronous network model. Next, we review the asynchronous protocols and roughly categorize them according to their batch-processing capabilities.

Non-batched asynchronous protocols. Asynchronous proactive secret sharing protocol (APSS) was initially proposed in CKL+02 [23]. In this construction, each node reshapes its share of the secret through an independent secret sharing instance, after which all nodes invoke a validated Byzantine agreement to decide which instances completed successfully. The communication complexity of CKL+02 [23] is $O(\kappa n^4)$. ZSR05 [78] introduces ADPSS, the first dynamic-committee APSS protocol, but its communication complexity is exponential. Shanrang [73] reduces the cost to $O(\kappa n^4)$; however, it lowers the resilience to $1/4$ by adopting a CHURP-style technique [61], where the shares are refreshed via zero secret sharing. LongLive [77] and DyCAPS [54] reduce the communication cost to $O(\kappa n^3)$. DyCAPS [54] follows the ADPSS framework [61] and uses zero-secret sharing, whereas

LongLive builds a high-threshold variant of ADPSS inspired by CKL+02 [23], raising the reconstruction threshold to $2n/3$. Opt-DPSS [53] reaches $O(\kappa n^2)$ communication complexity in the optimistic setting where all participants behave honestly, and FGL+25 [41] attains the same $O(\kappa n^2)$ communication cost under the worst-case bound by combining a fixed-size reference committee with public verifiable secret sharing, where the sharing can be verified by external users.

Batched asynchronous protocols. LongLive [77] introduces the first batched ADPSS protocol, which follows the MPC-based paradigm outlined in Section 3.2. In the high-threshold setting, it attains a per-secret communication complexity of $O(\kappa n^2)$ by employing Pedersen commitment. In the low-threshold setting, the per-secret cost drops to the optimal $O(\kappa n)$, provided a trusted setup for KZG commitment [57]. FGL+25 [41] and Opt-DPSS [53] extend the same MPC-based approach to their own batched variants. Specifically, the batched Opt-DPSS [53] incurs a communication complexity of $O(\kappa n^2)$ per secret in both high-threshold and low-threshold settings, while improving the complexity to $O(\kappa n)$ in the optimistic setting with KZG commitment [57].

B Epoch Definition

ABK+22 [3] proved an impossibility result that if a mobile adversary can eventually compromise an unlimited number of shareholders, no threshold protocol can preserve the secret indefinitely. To sidestep this barrier, we adopt the epoch-based model introduced by MPSS [64] and further improved in LongLive [77] to constrain the power of the mobile adversary. In the epoch-based model, the long-lived system executes in discrete periods, termed epochs, and the adversary is assumed to corrupt at most t parties within any single epoch. Since an asynchronous network offers no global clock or timing assumptions, the definition of an epoch is event-driven and described as follows.

Local epoch. A node is in epoch e if it retains any secret shares or keys associated with epoch e , including any reshares derived during the resharing procedure. The node transitions from epoch e to epoch $e+1$ only after it irreversibly erases all such material, including original shares, keys, and locally stored reshares, from both volatile and persistent storage, making subsequent recovery infeasible. A node that initiates resharing while still holding epoch e shares is regarded as belonging to epochs e and $e+1$ simultaneously until resharing materials have been received from all honest parties and both the prior shares and resharing materials are securely deleted.

Global epoch. Let G_e denote the set of nodes responsible for the secrets in epoch e . The system is in system epoch e from the moment the first honest node in G_e enters epoch e until the moment the last honest node in G_e leaves epoch e .

C Supplementary for Building Blocks

Reliable broadcast. A reliable broadcast protocol ensures that a message m distributed by a dealer $\mathcal{P}$ is consistently received by all participants. We adopt the construction from DXR21 [35], which achieves a communication complexity of $O(n|m| + \kappa n^2)$ when executed among n nodes. A reliable broadcast protocol satisfies the following properties:

- *Correctness:* All honest nodes that output a message output the same one. If the dealer $\mathcal{P}$ is honest, the output message is the original input of $\mathcal{P}$.
- *Completeness:* If any honest node outputs a message, or if an honest dealer inputs a message, all honest nodes output a message eventually.

Multi-valued validated asynchronous byzantine agreement (MVBA). MVBA is an asynchronous Byzantine fault-tolerant consensus protocol, initially proposed in CKPS01 [24]. In MVBA, each participating node proposes an input value, the protocol ensures agreement on one of these values that satisfies a predefined common predicate. An MVBA protocol satisfies the following properties:

- *Agreement:* All honest nodes output the same value.
- *Termination:* If all honest nodes propose inputs that satisfy the predicate, then all honest nodes eventually output.
- *External Validity:* If an honest node outputs a value, then the output satisfies the predicate.

One-sided voting (OSV). One-sided voting is a weak agreement primitive in which each node independently decides whether to endorse a given event, and the participation of the protocol is counted as a vote. A successful decision is reached once at least $n - t$ nodes honestly participate. OSV is instantiated using the Bracha broadcast [21], omitting the proposal phase. This protocol terminates in $O(1)$ round and incurs a communication complexity of $O(\kappa n^2)$. An (n, t) -OSV protocol satisfies the following properties if whenever at most t -out-of- n nodes are corrupted:

- *Correctness:* If an honest node outputs done, then at least $n - 2t$ honest nodes initiated the protocol.
- *Completeness:* If some honest nodes output done or all honest nodes initiate it, then all honest nodes output eventually.

D Supplementary for GoSSamer-ACSS's Applications

Random beacon. Random beacon [1, 2, 18] enables the distributed generation of random values, and is widely used in asynchronous consensus [39], secure multi-party computation [31], distributed noise generation [40], and committee election in sharding blockchains [58] or Proof-of-Stake blockchains [45]. Batched asynchronous complete secret sharing, or asynchronous verifiable secret sharing, is an important

building block in random beacon. To the best of our knowledge, the most efficient constructions in random beacon include: the scheme BBB+24 [9], which is computationally lightweight but incurs a communication cost of $O(\kappa n^3 \log n)$ per beacon; the scheme Bingo [1], which achieves $O(\kappa n^2)$ per-beacon communication cost but requires expensive group exponentiations and a trusted setup. By adopting GoSSamer-ACSS as a drop-in replacement for the underlying ACSS implementation in [9], we can construct an efficient random beacon protocol with both $O(\kappa n^2)$ per-beacon communication and lightweight computation with only lightweight cryptographic primitives.

Distributed key generation. Distributed key generation enables the distributed generation of threshold public and private key pairs among mutually distrustful nodes, which utilizes (batched) ACSS to facilitate the distributed randomness generation with completeness. The best-known results in asynchronous distributed key generation either derive keys from lightweight ACSS with an amortized communication complexity of $O(\kappa n^3)$ under Byzantine faults [50], or derive keys from expensive commitment-based ACSS with an amortized communication complexity of $O(\kappa n^3)$ [34, 36] or $O(\kappa n^2 \log n)$ [7]. By adopting GoSSamer-ACSS as a drop-in replacement for the underlying ACSS implementation in [50], we can construct an efficient asynchronous distributed key generation protocol, which derives keys from lightweight ACSS schemes and achieves an amortized communication complexity of $O(\kappa n^2)$.

Multi-party computation. Multi-party computation enables mutually distrustful parties to jointly compute a function over their private data. Batched ACSS serves as a foundational primitive for robustness in many asynchronous multi-party computation protocols [12, 48, 63]. To the best of our knowledge, the most efficient constructions in asynchronous multi-party computation include: the scheme CP23 [29] incurs a communication complexity of $O(\kappa n^4)$ per sharing; the scheme GLS24 [48] achieves the optimal per-sharing communication complexity of $O(\kappa n)$, but its total complexity reaches $O(\kappa b n + \kappa^2 n^{14})$. GoSSamer-ACSS provides a concretely efficient alternative for building a lightweight asynchronous multi-party computation protocol with $O(\kappa n)$ per-sharing communication and $O(\kappa b n + \kappa n^4 \log n)$ total communication.

E Analysis of GoSSamer-ACSS

E.1 Performance Analysis

Theorem 1. Consider a committee of n nodes, where up to $t < n/3$ are corrupted. Let the batch size be $b = \text{poly}(n)$. GoSSamer-ACSS implements batched ACSS with a total communication complexity of $O(\kappa b n + \kappa n^3 \log n)$ and an amortized communication complexity of $O(\kappa n)$ per secret.

Proof of Theorem 1. Let $O_{\text{dist}}(x)$ denote the communication complexity of using secure message distribution to distribute the message $(m_1, \dots, m_n)$ of length x to n participating nodes,

such that each node $\mathcal{P}_i$ receives its designated message m_i . Let O_{osv} denote the communication of a one-sided voting instance, and let $O_{\text{rbc}}(x)$ denote the communication complexity of reliably broadcasting a message of length x . As discussed in Section 2.3, there is: $O_{\text{dist}}(x) = O(x + \kappa n^2 \log n)$, $O_{\text{osv}} = O(\kappa n^2)$, and $O_{\text{rbc}}(x) = O(n x + \kappa n^2)$. Let $m = b/(t+1)$. The expected total communication complexity in Phase 1 and Phase 2 is given by:

$$\begin{aligned} & n O_{\text{dist}}(\kappa n m) + O_{\text{rbc}}(\kappa n^2) + O_{\text{osv}} \\ &= O(\kappa b n + \kappa n^3 \log n + \kappa n^3 + \kappa n^2) \end{aligned}$$

Let $O_{\text{forw}}(x)$ denote the communication complexity of using secure message distribution to forward a message of length x to a node. As defined in Section 2.3, there is $O_{\text{forw}}(x) = O(x + \kappa n \log n)$. The expected total communication complexity in Phase 3 and Phase 4 is given by:

$$\begin{aligned} & n^2 O_{\text{forw}}(\kappa m) + n^2 O_{\text{forw}}(\kappa m) + O(\kappa m n^2) \\ &= O(\kappa b n + \kappa n^3 \log n) \end{aligned}$$

Thus, the total asymptotic communication complexity of GoSSamer-ACSS is $O(\kappa b n + \kappa n^3 \log n)$. When the batch size asymptotically satisfies $b = \text{poly}(n)$ and $b \geq O(n^2 \log n)$, the amortized communication complexity is $O(\kappa n)$ per secret. $\square$

E.2 Security Analysis

Lemma 1 (Correctness). GoSSamer-ACSS satisfies the correctness property of ACSS.

Proof of Lemma 1. The correctness property of ACSS requires that, given an honest dealer $\mathcal{P}$ who inputs b secrets $\mathbf{a}$, there exists a vector of b polynomials $\mathbf{f}(x) \in \mathbb{Z}_p[x]_{\leq t}^b$ such that $\mathbf{f}(0) = \mathbf{a}$, and each honest node $\mathcal{P}_i$ outputs shares $[\mathbf{a}]_i = \mathbf{f}(i)$.

In GoSSamer-ACSS, an honest dealer encodes $\mathbf{a}$ following Lines 1-5 of Algorithm 1. Specifically, the secrets are partitioned into $m = b/(t+1)$ groups, each of size $t+1$. For each group $\mathbf{a}_u$, a t -degree polynomial $f_u(x)$ is interpolated. All interpolated polynomials are then encoded into a vector of m bivariate polynomials $\mathbf{A}(x, y)$, where each $A_u(x, y)$ satisfies $A_u(x, 0) = f_u(x)$. For any secret $\mathbf{a}_{u,v}$ (the v -th secret in group $\mathbf{a}_u$), the following equation holds:

$$A_u(v, 0) = f_u(v) = \mathbf{a}_{u,v}$$

This further implies that $A_u(v, y)$ is a t -degree univariate secret sharing polynomial for the secret value $\mathbf{a}_{u,v}$.

Following Lines 7-12 of Algorithm 1, the honest dealer distributes the evaluations $\mathbf{A}(1, \ell), \dots, \mathbf{A}(n, \ell)$ and the evaluations $E(1, \ell), \dots, E(n, \ell)$ to each node $\mathcal{P}_\ell$ via secure message distribution, and broadcasts a verification script $v(x, y)$. Each

node $\mathcal{P}_i$ verifies the correctness condition of each received evaluation $\mathbf{A}(k, i)$:

$$v(k, i) = \mathbf{r} \cdot \mathbf{A}(k, i) + E(k, i)$$

This condition always holds when the dealer is honest. Thus, all honest nodes proceed to one-sided voting. Guaranteed by its completeness, every honest node eventually outputs in voting. Each node $\mathcal{P}_i$ then computes its shares by interpolating the received evaluations into a t -degree polynomial $\mathbf{A}(x, i)$, and reverting the dealer's encoding process to obtain $m \cdot (t + 1) = b$ values, which together form the output share $[\mathbf{a}]_i$.

Let $\mathbf{h}(x)$ be the vector of b polynomials required by the correctness definition. For each secret $s_\tau \in \mathbf{a}$ that corresponds to some $\mathbf{a}_{\tilde{u}, \tilde{v}}$, define $h_\tau(y) = A_{\tilde{u}}(\tilde{v}, y)$. Each h_τ is of degree t and satisfies $h_\tau(0) = s_\tau$, so that $\mathbf{h}(0) = \mathbf{a}$. Each honest node $\mathcal{P}_i$ outputs $[\mathbf{a}]_i = \mathbf{h}(i)$. Therefore, GoSSamer-ACSS satisfies the correctness property. $\square$

Lemma 2 (Schwartz-Zippel Lemma). Let $f \in \mathbb{Z}_p[x_1, \dots, x_n]$ be a multivariate polynomial over $\mathbb{Z}_p$ with degree at most t . Let $r_1, \dots, r_n$ be chosen independently and uniformly from $\mathbb{Z}_p$. Then, $\Pr[f(r_1, \dots, r_n) = 0] \leq t/|\mathbb{Z}_p|$.

Lemma 3 (Completeness). Assuming that 2^{-n} is negligible and the field size satisfies $|\mathbb{Z}_p| = 2^{\text{poly}(n)} > 2^{2n}$, GoSSamer-ACSS satisfies the completeness property of ACSS.

Proof of Lemma 3. The completeness property of ACSS requires that if any honest node outputs, there must exist a vector of b polynomials $\tilde{\mathbf{f}}(x) \in \mathbb{Z}_p[x]_{\leq t}^b$ such that every honest node $\mathcal{P}_i$ eventually outputs $\tilde{\mathbf{f}}(i)$.

If an honest node outputs, it must have received its valid share and output in the one-sided voting. By the correctness of one-sided voting, at least $n - 2t$ honest nodes have initiated the voting protocol. For each such node $\mathcal{P}_i$, the received share $\mathbf{A}(x, i)$, blinding share $E(x, i)$, and verification script $v(x, y)$ must satisfy the following condition

$$v(x, i) = \mathbf{r} \cdot \mathbf{A}(x, i) + E(x, i)$$

The remaining t honest nodes either later receive valid shares or fail in the verification due to dealer misbehavior. In the latter case, the share $\tilde{\mathbf{A}}(x, i)$ of $\mathcal{P}_i$ is invalid due to:

$$v(x, i) \neq \tilde{v}(x, i) := \mathbf{r} \cdot \tilde{\mathbf{A}}(x, i) + \tilde{E}(x, i)$$

Each node with an invalid share initiates Phase 3 of Algorithm 2, and forwards a complain message to all nodes with a pair of faulty evaluations $(\tilde{\mathbf{A}}(e, i), \tilde{E}(e, i))$, which satisfies the following equation. Eventually, all honest nodes confirm the malicious dealer and enter the following recovery phase.

$$v(e, i) \neq \tilde{v}(e, i) := \mathbf{r} \cdot \tilde{\mathbf{A}}(e, i) + \tilde{E}(e, i)$$

In Phase 4 of Algorithm 2, nodes perform share recovery. Each honest node $\mathcal{P}_i$ with valid share $\mathbf{A}(x, i)$ and $E(x, i)$ forwards evaluation $(\mathbf{A}(u, i), E(u, i))$ to each node $\mathcal{P}_u$. As at least

$t + 1$ honest nodes possess valid shares, each node $\mathcal{P}_u$ can collect $\mathbf{A}(u, i)$ from at least $(t + 1)$ honest nodes $\mathcal{P}_i$, where

$$v(u, i) = \mathbf{r} \cdot \mathbf{A}(u, i) + E(u, i)$$

The recipient $\mathcal{P}_u$ can interpolate the valid evaluations into a t -degree column polynomial $\mathbf{A}(u, y)$, and further distribute each $\mathbf{A}(u, v)$ to each node $\mathcal{P}_v$. As all honest nodes eventually reconstruct the column polynomial, every node $\mathcal{P}_v$ can collect at least $n - t$ valid evaluations. In such a case, $\mathcal{P}_v$ can perform the robust polynomial interpolation to reconstruct $(\mathbf{A}(x, v), E(x, v))$, decode $\mathbf{A}(x, v)$ into its local secret share $[\mathbf{a}]_v$, and output accordingly. It can be intuitively proved that:

$$v(x, v) = \mathbf{r} \cdot \mathbf{A}(x, v) + E(x, v)$$

It remains to show that there exists polynomials $\tilde{\mathbf{f}}(x) \in \mathbb{Z}_p[x]_{\leq t}^b$ such that each honest node $\mathcal{P}_i$ outputs $\tilde{\mathbf{f}}(i)$. Let $\mathbf{A}(x, y)$ be the vector of secret sharing polynomials interpolated from the share-encoding polynomial $\mathbf{A}(x, i)$ from any $(t + 1)$ honest nodes $\mathcal{P}_i$. Consider another honest node $\mathcal{P}_u$ that outputs $\tilde{\mathbf{A}}(x, u)$ and $\tilde{E}(x, u)$, the following equation holds as proved above.

$$v(x, u) = \mathbf{r} \cdot \mathbf{A}(x, u) + E(x, u) = \mathbf{r} \cdot \tilde{\mathbf{A}}(x, u) + \tilde{E}(x, u)$$

This implies that if $\mathbf{A}(x, u) \neq \tilde{\mathbf{A}}(x, u)$, then $\mathbf{r}$ must satisfy the following equation.

$$\mathbf{r} \cdot (\mathbf{A}(x, u) - \tilde{\mathbf{A}}(x, u)) + (E(x, u) - \tilde{E}(x, u)) = 0$$

According to Lemma 2, for any subset of at most $n - t$ honest nodes, the probability of the above event is at most $1/|\mathbb{Z}_p|$. By applying a union bound over all these subsets, the total error probability becomes $2^n/|\mathbb{Z}_p|$, which is negligible when $|\mathbb{Z}_p| > 2^{2n}$. Finally, by Lemma 1, $\mathbf{A}(x, u)$ of $\mathcal{P}_u$ can be decoded into $\tilde{\mathbf{f}}(u)$. This concludes the proof that GoSSamer-ACSS satisfies completeness, except with negligible probability. $\square$

Lemma 4 (Secrecy). GoSSamer-ACSS satisfies the secrecy property of ACSS. Specifically, there exists a P.P.T simulator $\mathcal{S}_{\text{acss}}$ which, given the shares of corrupted nodes and a vector of independently sampled secrets, can produce a simulated view in the ideal world, such that the ideal and real worlds are indistinguishable from the view of $\mathcal{A}$.

Proof of Lemma 4. We employ a hybrid argument to prove that the view of the adversary $\mathcal{A}$ in the real protocol execution with actual secrets is indistinguishable from its view in the ideal world. In the ideal world, the simulator $\mathcal{S}_{\text{acss}}$ is given only the secret shares held by corrupted nodes and simulates with uniformly sampled secrets, as summarized in Figure 7.

Hybrid 0. This is a real-world execution of GoSSamer-ACSS. The honest dealer $\mathcal{P}$ inputs secrets $\mathbf{a}$ and all honest nodes adhere to the protocol.

<u>Simulator $\mathcal{S}_{\text{acss}}$</u>
input: (n, t) , batch size b , field $\mathbb{Z}_p$, corrupted node set $\mathcal{C} \subseteq [n]$ and $ \mathcal{C} \leq t$, secret shares $[\mathbf{a}]_i$ of b secrets $\mathbf{a}$ of corrupted nodes $\mathcal{P}_i \in \mathcal{C}$ notations: the number of groups $m = b/(t+1)$
1: partition $[\mathbf{a}]_i$ into m groups of size $t+1$, interpolate each group, and obtain a vector of m polynomials $\mathbf{A}(x, i)$ of t -degree for $i \in \mathcal{C}$ 2: uniformly sample a vector of m polynomials $\tilde{\mathbf{A}}(x, y)$ with t -degree, such that $\tilde{\mathbf{A}}(x, i) := \mathbf{A}(x, i)$ for $i \in \mathcal{C}$ 3: uniformly sample m -dimension challenge vector $\tilde{\mathbf{r}} \leftarrow \mathbb{Z}_p^m$ 4: uniformly sample t -degree polynomial $\tilde{v}(x, y)$ as the verification script 5: calculate blinding polynomial as $\tilde{E}(x, y) := \tilde{v}(x, y) - \tilde{\mathbf{r}} \cdot \tilde{\mathbf{A}}(x, y)$ 6: calculate $\mathbf{rt}$ from secure message distribution, and program the random oracle to output $\tilde{\mathbf{r}}$ on input $\mathbf{rt}$ 7: distribute points $(\tilde{\mathbf{A}}(1, j), \tilde{E}(1, j)), \dots, (\tilde{\mathbf{A}}(n, j), \tilde{E}(n, j))$ to each node $\mathcal{P}_j$ for $j \in [n]$ through secure message distribution 8: simulate the rest of the protocol on behalf of all honest nodes

Figure 7: The simulator $\mathcal{S}_{\text{acss}}$ of GoSSamer-ACSS

Hybrid 1. Same as Hybrid 0, except that the honest nodes are simulated and honestly follow the protocol. In simulation, let $\mathbf{a}$ be a vector of randomly sampled secrets, $\mathbf{A}(x, y)$ be the randomly sampled secret sharing polynomials with $\mathbf{A}(x, 0)$ encodes $\mathbf{a}$, $E(x, y)$ be the randomly sampled blinding polynomial and $v(x, y) = \mathbf{r} \cdot \mathbf{A}(x, y) + E(x, y)$ be the verification script, where $\mathbf{r}$ is the challenge vector. Hybrid 1 is indistinguishable from Hybrid 0 since all distributions remain identical.

Hybrid 2. Same as Hybrid 1, except that all honest nodes are simulated by the simulator $\mathcal{S}_{\text{acss}}$ described in Figure 7. Specifically, the simulator replaces the real secret sharing polynomials $\mathbf{A}(x, y)$ with independently and uniformly sampled polynomials $\tilde{\mathbf{A}}(x, y)$, which encode uniformly sampled secrets $\tilde{\mathbf{A}}(x, 0)$ in place of real secrets $\mathbf{A}(x, 0)$. For any corrupted node $\mathcal{P}_i$, the simulator ensures that its share-encoding polynomials $\tilde{\mathbf{A}}(x, i)$ remain the same as $\mathbf{A}(x, i)$. Subsequently, the simulator randomly samples a verification script $\tilde{v}(x, y)$ that is identically distributed to $v(x, y)$ in the real protocol. The blinding polynomial $\tilde{E}(x, y)$ is thus computed as:

$$\tilde{E}(x, y) := \tilde{v}(x, y) - \tilde{\mathbf{r}} \cdot \tilde{\mathbf{A}}(x, y)$$

Because the underlying secure message distribution protocol ensures secrecy, the only difference between Hybrid 2 and Hybrid 1 arises from the $\tilde{E}(x, i)$ and $E(x, i)$ observed by each corrupted node $\mathcal{P}_i$. If the hash function used to generate $\mathbf{r}$ is modeled as a random oracle, then $\mathbf{r}$ is indistinguishable from a randomly sampled vector. As a result, the distributions of $\tilde{E}(x, i)$ and $E(x, i)$ are indistinguishable, and therefore Hybrid 2 is indistinguishable from Hybrid 1. The sequence of indistinguishable hybrids implies that the adversary cannot distinguish between real and ideal executions. Hence, GoSSamer-ACSS satisfies the secrecy property. $\square$

Theorem 2. GoSSamer-ACSS is an ACSS protocol that satisfies correctness, completeness and secrecy against any P.P.T Byzantine adversary corrupting up to $t < n/3$ nodes.

Proof of Theorem 2. The theorem follows directly from Lemma 1, Lemma 3, and Lemma 4. $\square$

Proof Strategy. We prove the security of our GoSSamer-ACSS and GoSSamer-ADPSS protocols in the stand-alone setting, following a well-established line of work [23, 34, 36, 76]. For the secrecy property, we construct a simulator that, given only the shares held by corrupted nodes, replaces the original secrets with uniformly random values in the ideal world and produces a simulated view that is indistinguishable from the real protocol execution. Although our analysis is conducted in the standalone setting, we believe that the use of a straight-line, non-rewinding simulator provides a promising foundation for extending our proof to the universal composability (UC) framework [26]. However, several technical challenges prevent us from presenting a UC-based proof at this time. First, some building blocks used in our constructions, such as secure message distribution, are not readily captured by UC-secure formulations for certain properties, such as completeness, making a direct UC proof challenging without extensive modeling. Second, as discussed in [42], standard UC composition theorems do not apply directly in asynchronous settings, further complicating modular UC analysis. Thus, we leave the fully UC-secure formulations and proofs to future work.

F Analysis of Dual-Committee ACSS

F.1 Definition

DC-ACSS: (batched) dual-committee ACSS. Consider two committees, com and com' , whose parameters are given by (n, t) and (n', t') , respectively, where n (resp., n') denotes the committee size and t (resp., t') the corresponding threshold. DC-ACSS executes an (n, t, b) -ACSS among com and an (n', t', b) -ACSS among com' . Both ACSS instances have the same dealer $\mathcal{P}$ and share an identical vector of b input secrets, denoted as $\mathbf{a} \leftarrow \mathbb{Z}_p^b$. A dual-committee ACSS protocol satisfies the following properties:

- **Correctness:** If the dealer $\mathcal{P}$ is honest and inputs the vector of secrets $\mathbf{a}$, then there exist two batches of b polynomials $\mathbf{f}(x)$ and $\mathbf{f}(x)'$ in $\mathbb{Z}_p[x]_{\leq t}$, each of degree at most t , such that $\mathbf{f}(0) = \mathbf{f}(0)' = \mathbf{a}$. Every honest node $\mathcal{P}_i$ in com outputs the share $[\mathbf{a}]_i = \mathbf{f}(i)$, and every honest node $\mathcal{P}_i'$ in com' outputs the share $[\mathbf{a}]_i = \mathbf{f}(i)'$.
- **Completeness:** If any honest node outputs, then there exists polynomials $\tilde{\mathbf{f}}(x), \tilde{\mathbf{f}}(x)' \in \mathbb{Z}_p[x]_{\leq t}$ with $\tilde{\mathbf{f}}(0) = \tilde{\mathbf{f}}(0)'$, such that every honest node $\mathcal{P}_i$ in com eventually outputs $\tilde{\mathbf{f}}(i)$ and every honest node $\mathcal{P}_i'$ in com' eventually outputs $\tilde{\mathbf{f}}(i)'$.
- **Secrecy:** If the dealer is honest, DC-ACSS reveals no information about $\mathbf{a}$ to the adversary. Formally, there exists a P.P.T simulator $\mathcal{S}_{\text{dc-acss}}$ such that, given the secret shares of corrupted nodes and an independently sampled vector of secrets, can produce a simulated view in the ideal world,

such that the ideal and real worlds are indistinguishable from the view of $\mathcal{A}$.

F.2 Theoretical Analysis

Lemma 5 (Correctness). The lightweight dual-committee ACSS protocol described in Lines 1-8 of Algorithm 4 satisfies the correctness property of dual-committee ACSS.

Proof of Lemma 5. The correctness property follows directly from the correctness of the underlying ACSS protocol, as established in Lemma 1. $\square$

Lemma 6 (Completeness). The lightweight dual-committee ACSS protocol described in Lines 1-8 of Algorithm 4 satisfies the completeness property of dual-committee ACSS.

Proof of Lemma 6. The completeness of dual-committee ACSS requires that if any honest node outputs, then there exists polynomials $\tilde{\mathbf{f}}(x), \tilde{\mathbf{f}}(x)' \in \mathbb{Z}_p[x]_{\leq t}$ with $\tilde{\mathbf{f}}(0) = \tilde{\mathbf{f}}(0)'$, such that every honest node $\mathcal{P}_i$ in com eventually outputs $\tilde{\mathbf{f}}(i)$ and every honest node $\mathcal{P}'_i$ in com' eventually outputs $\tilde{\mathbf{f}}(i)'$.

First, we prove that if an honest node outputs, then all honest nodes in both committees output.

Case 1: an honest node in com outputs its share.

If an honest node in committee com produces an output in the protocol, it must have received $t + 1$ identical values of $\bar{w}(0)$ from committee com', and output $w(x)$ from the robust polynomial interpolation algorithm, where $\bar{w}(0) = w(0)$. Receiving $t + 1$ identical copies of $\bar{w}(0)$ means that at least one honest node in com' must have multicast this value. An honest node multicasts $\bar{w}(0)$ only after it has reconstructed $\bar{w}(x)$ via the robust polynomial interpolation algorithm, and that reconstruction is possible only after the node has obtained at least $n' - t'$ valid shares. Therefore, at least $t' + 1$ honest nodes of com' must already output in ACSS and multicast their shares $[\bar{w}]_i$ within com'. By the completeness of ACSS, if an honest node outputs its share, all honest nodes in the committee com' eventually output their shares. Similarly, if an honest node in the committee com outputs $w(x)$ from the robust polynomial interpolation algorithm, all honest nodes in the committee com output their shares in ACSS. Therefore, if an honest node in the committee com outputs in the dual-committee ACSS protocol, all honest nodes in both committees eventually output in the underlying ACSS protocol.

Intuitively follows the completeness of ACSS, all honest nodes in com will output the same $w(x)$ from robust polynomial interpolation, and all honest nodes in com' output the same $\bar{w}(x)$. After multicasting $\bar{w}(0)$ from the committee com' to com, and multicasting $w(0)$ from the committee com to com', all nodes in each committee receive at least $(n - t)$ identical evaluations from the other committee, ensuring that all honest nodes in both committees agree on $w(0)$ and $\bar{w}(0)$. Thus, if any honest node observes $w(0) = \bar{w}(0)$ and outputs, it

follows that all honest nodes in both committees will eventually agree on $w(0) = \bar{w}(0)$ and output as well.

Case 2: an honest node in com' outputs its share.

The proof for this case is analogous to that of Case 1.

Next, we prove that there exists polynomials $\tilde{\mathbf{f}}(x), \tilde{\mathbf{f}}(x)' \in \mathbb{Z}_p[x]_{\leq t}$ with $\tilde{\mathbf{f}}(0) = \tilde{\mathbf{f}}(0)'$, such that the output share is $\tilde{\mathbf{f}}(i)$ for every honest node $\mathcal{P}_i$ in com, and $\tilde{\mathbf{f}}(i)'$ for every honest node $\mathcal{P}'_i$ in com'. The above argument proves that $w(0) = \bar{w}(0)$, where $w(x)$ and $\bar{w}(x)$ are output from robust polynomial interpolation, and at least $n - t$ evaluations on $w(x)$ and $n' - t'$ evaluations on $\bar{w}(x)$ satisfy the following equation:

$$w(i) = [w]_i = \mathbf{r} \cdot [\mathbf{a}]_i + [q]_i, \bar{w}(i) = [\bar{w}]_i = \mathbf{r} \cdot [\bar{\mathbf{a}}]_i + [\bar{q}]_i$$

Therefore, it holds that:

$$\mathbf{r} \cdot \mathbf{a} + q = \mathbf{r} \cdot \bar{\mathbf{a}} + \bar{q}$$

Since $\mathbf{r}$ is chosen uniformly at random, the above equation implies that if $\mathbf{a} \neq \bar{\mathbf{a}}$, $\mathbf{r}$ must satisfy

$$\mathbf{r} \cdot (\mathbf{a} - \bar{\mathbf{a}}) + q - \bar{q} = 0$$

By Lemma 2, the probability of the above event is at most $1/|\mathbb{Z}_p|$, which is negligible when $|\mathbb{Z}_p| = 2^{\text{poly}(n)}$. Thus, it can be proved that $\mathbf{a} = \bar{\mathbf{a}}$, except with negligible probability. Moreover, the completeness property of ACSS guarantees the existence of t -degree polynomials $(\mathbf{a}(x), q(x))$ and $(\bar{\mathbf{a}}(x), \bar{q}(x))$ with $\mathbf{a}(0) = \mathbf{a}$ and $\bar{\mathbf{a}}(0) = \bar{\mathbf{a}}$, such that each honest node $\mathcal{P}_i$ in committee com outputs $\mathbf{a}(i)$ and $q(i)$, and each honest node $\mathcal{P}'_i$ in committee com' outputs $\bar{\mathbf{a}}(i)$ and $\bar{q}(i)$. This concludes the proof that the protocol satisfies completeness. $\square$

Lemma 7 (Secrecy). The lightweight dual-committee ACSS protocol described in Lines 1-8 of Algorithm 4 satisfies the secrecy property of dual-committee ACSS.

Proof of Lemma 7. We employ the same proof strategy as in Lemma 4, utilizing a hybrid argument to demonstrate that the adversary's view, during the real execution with actual secrets is indistinguishable from its view in the ideal world. In the ideal world, the simulator $\mathcal{S}_{\text{dc-acs}}$ is given only the secret shares held by corrupted nodes and simulates the protocol using uniformly sampled secrets, as summarized in Figure 8.

Hybrid 0. Real-world protocol execution.

Hybrid 1. Same as Hybrid 0, except that all honest nodes are simulated and honestly follow the protocol. In simulation, the vector of secrets $\mathbf{a} \leftarrow \mathbb{Z}_p^b$ and the blinding secret $q \leftarrow \mathbb{Z}_p$ are randomly sampled. The tuple $(\mathbf{a}, q)$ is then independently shared with committees com and com' using two separate instances of the ACSS protocol. The indistinguishability between Hybrid 0 and Hybrid 1 follows from the fact that the distributions of $(\mathbf{a}, q)$ in both hybrids are identical.

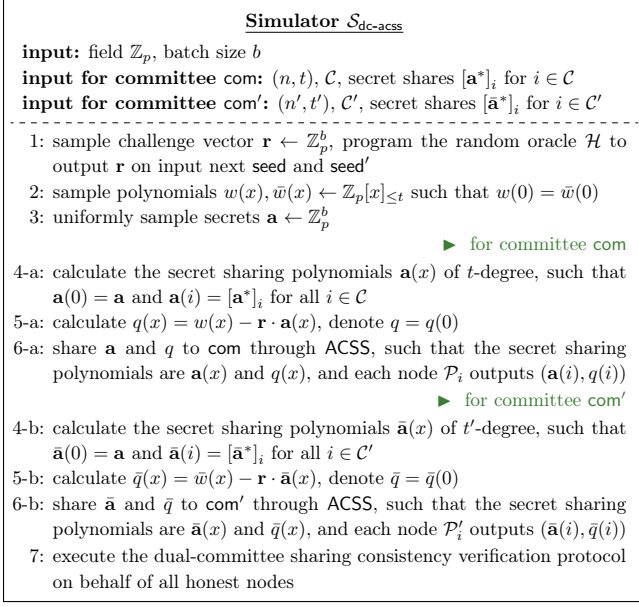


Figure 8: The simulator $\mathcal{S}_{\text{dc-acss}}$ of dual-committee ACSS

Hybrid 2. Same as Hybrid 1, except that all honest nodes are simulated by the simulator specified in Figure 8. Specifically, the simulator replaces the real secrets $\mathbf{a}^*$ with a randomly sampled secrets $\mathbf{a}$, while ensuring that the secret share received by each corrupted node remains unchanged. The simulator then randomly samples t -degree polynomials $w(x)$ and $\bar{w}(x)$ with $w(0) = \bar{w}(0)$, which are distributed identically to those in Hybrid 1. It calculates the secret sharing polynomials $q(x)$ and $\bar{q}(x)$ of the blinding secret using the following equations, where $\mathbf{r}$ is a randomly sampled challenge vector of b -dimension:

$$q(x) = w(x) - \mathbf{r} \cdot \mathbf{a}(x), \quad \bar{q}(x) = \bar{w}(x) - \mathbf{r} \cdot \bar{\mathbf{a}}(x)$$

The only difference between Hybrid 2 and Hybrid 1 arises from the value $\{q(i)\}_{i \in \mathcal{C}}$ and $\{\bar{q}(i)\}_{i \in \mathcal{C}'}$. By the secrecy of ACSS, additional information in the view of $\mathcal{A}$ is given by $w(0)$ and $\bar{w}(0)$. Because these values are randomly sampled, $q(0)$ is indistinguishable from random, $\{q(i)\}_{i \in \mathcal{C}}$ and $\{\bar{q}(i)\}_{i \in \mathcal{C}'}$ are indistinguishable from those in the real protocol execution, and thus Hybrid 2 is indistinguishable from Hybrid 1. The sequence of indistinguishable hybrids implies that the adversary cannot distinguish between the real and the ideal worlds. Hence, the protocol satisfies the secrecy property. $\square$

Theorem 3. The lightweight dual-committee ACSS protocol, as specified in Lines 1-8 of Algorithm 4, implements a dual-committee ACSS protocol that satisfies correctness, completeness, and secrecy against any P.P.T Byzantine adversaries that corrupt up to $t < n/3$ nodes.

Proof of Theorem 3. The theorem follows directly from Lemma 5, Lemma 6, and Lemma 7. $\square$

G Analysis of GoSSamer-ADPSS

G.1 Performance Analysis

Theorem 4. Consider a committee of n nodes, where up to $t < n/3$ are corrupted. Let the batch size be $b = \text{poly}(n)$. GoSSamer-ADPSS implements batched asynchronous dynamic proactive secret sharing with a total communication complexity of $O(\kappa bn + \kappa n^4 \log n)$ and an amortized communication complexity of $O(\kappa n)$ per secret.

Proof of Theorem 4. Let $O_{\text{acss}}(x)$ denote the total communication complexity of using GoSSamer-ACSS to share a batch of x secrets, and let O_{mvba} denote the communication complexity of MVBA. As discussed in Section 5.2 and proved in Theorem 1, there is: $O_{\text{mvba}} = O(\kappa n^3)$ and $O_{\text{acss}}(x) = O(\kappa xn + \kappa n^3 \log n)$. Let $m = b/(t+1)$. The expected communication complexity in GoSSamer-ADPSS is given by:

$$\begin{aligned} & nO_{\text{acss}}(m) + nO(\kappa n^2) + O_{\text{mvba}} + O(\kappa n^3) + O(\kappa n^2 m) \\ &= O(\kappa bn + \kappa n^4 \log n) \end{aligned}$$

Thus, the total asymptotic communication complexity of GoSSamer-ADPSS is $O(\kappa bn + \kappa n^4 \log n)$. When the batch size asymptotically satisfies $b = \text{poly}(n)$ and $b \geq O(n^3 \log n)$, the amortized communication is $O(\kappa n)$ per secret. $\square$

Asymptotically optimal per-secret communication with practical bandwidth saving. The best-known results [77] in prior ADPSS protocols attain linear per-secret communication within a single committee, yet still incur quadratic cost across committees. GoSSamer-ADPSS closes this gap: it achieves the information-theoretic lower bound of $O(\kappa n)$ communication per secret both inside and across committees. Although the asymptotical analysis indicates that the per-secret communication in GoSSamer-ADPSS becomes linear only for batch sizes $b \geq O(n^3 \log n)$, the practical evaluations demonstrate that GoSSamer-ADPSS already yields substantial bandwidth savings under more realistic parameters. Specifically, we fix the batch size to $k = n^2 \log n$. Asymptotically, this choice yields a per-secret communication complexity of $O(\kappa n^2)$ for GoSSamer-ADPSS, matching that of the state-of-the-art LongLive [77] protocol. For committee sizes $n = 4, 7, 16, 31$, the corresponding batch sizes are 32, 147, 1020, 4807, respectively. In a practical distributed key management service, such batch sizes cover periodic key resharing for fewer than 5000 users. Even in this constrained setting, GoSSamer-ADPSS demonstrates the lowest communication overhead, followed by LongLive-Ped and LongLive-KZG, as shown in Figure 5(c).

G.2 Security Analysis

Lemma 8 (Termination). GoSSamer-ADPSS satisfies the termination property of ADPSS.

Proof of Lemma 8. The termination property requires that if all honest nodes in the old committee com initiate the protocol, all honest nodes in the new committee com' output eventually.

Argument 1: all honest nodes in both committees can extract the share of the random masks.

The correctness property of the dual-committee ACSS ensures that if an honest node initiates an instance of the dual-committee ACSS, then all honest nodes in both committees will eventually output in this instance. Hence, if all honest nodes in com initiate the protocol, all honest nodes in both committees eventually output in all dual-committee ACSS instances initiated by the honest nodes, totaling at least $(n - t)$ instances. Consequently, all honest nodes in the old committee proceed to execute MVBA, where its termination, agreement, and external validity properties guarantee that all honest nodes output the common set Q_{ss} from MVBA, where all ACSS instances in Q_{ss} output eventually. Q_{ss} is then multicast from the old committee to the new committee com' , where each node receives at least $n - t$ identical Q_{ss} values. Thus, each node in both committees can locally extract the share of the random masks through the output from the dual-committee ACSS instances in Q_{ss} .

Argument 2: all honest nodes in the new committee can obtain the masked secrets.

Each honest node $\mathcal{P}_i$ in the old committee then encodes the masked share $[s]_i + [u]_i$ into the polynomial $F(x, i)$, and transmits the evaluation $F(u, i)$ to every node $\mathcal{P}_u$ in its committee. After collecting such evaluations, $\mathcal{P}_u$ invokes the robust polynomial interpolation algorithm. By the completeness of the dual-committee ACSS, this algorithm outputs $F(u, y)$ to $\mathcal{P}_u$. The node then multicasts $F(u, 0)$ to the new committee. As a result, all honest nodes in the new committee obtain $F(x, 0)$ by invoking the robust polynomial interpolation algorithm, from which the masked secrets $s + u$ can be decoded.

Given the above arguments, all honest nodes in the new committee com' can produce their secret shares eventually. Hence, the protocol satisfies the termination property. $\square$

Lemma 9 (Correctness). GoSSamer-ADPSS satisfies the correctness property of ADPSS.

Proof of Lemma 9. The correctness property of ADPSS requires that if a node $\mathcal{P}'_i$ in the new committee com' outputs the share $[\bar{s}]_i$, then all honest nodes in com' eventually output their shares, and there exists a vector of polynomials $\bar{f}(x)$ of degree t' such that $\bar{f}(0) = s$ and the share $[\bar{s}]_i = \bar{f}(i)$ for each node $\mathcal{P}'_i$. This holds under the assumption that there exists a vector of t -degree polynomials $f(x)$ satisfying $f(0) = s$, where each node $\mathcal{P}_i$ in the old committee holds the share $[s]_i = f(i)$.

Argument 1: all honest nodes can obtain the MVBA output and the corresponding ACSS output.

An honest node in the new committee com' outputs only if the robust polynomial interpolation algorithm outputs $F(x, 0)$, which indicates that at least $t + 1$ honest nodes in the old committee com have multicast $F(u, 0)$ as node $\mathcal{P}_u$. Since node $\mathcal{P}_u$ obtains the corresponding polynomial $F(u, y)$ through robust polynomial interpolation, at least $t + 1$ honest nodes $\mathcal{P}_i$ have multicast the evaluations on the polynomial $F(x, i)$, which is interpolated from the i -th share of the random masks. This further indicates that at least $t + 1$ honest nodes have obtained Q_{ss} in MVBA and outputted in all dual-committee ACSS instances in Q_{ss} . The agreement and termination properties of MVBA guarantee that all honest nodes in com output Q_{ss} , while the completeness property of the dual-committee ACSS ensures that all honest nodes in both committees output their shares in Q_{ss} .

Argument 2: the sharings of the random masks in the old and new committees are consistent.

The completeness of dual-committee ACSS guarantees that for all $j \in Q_{ss}$, there exists a vector of t -degree polynomials $\mathbf{a}_j(x)$ and a vector of t' -degree polynomials $\bar{\mathbf{a}}_j(x)$ with $\mathbf{a}_j(0) = \bar{\mathbf{a}}_j(0)$, such that each honest node $\mathcal{P}_i$ in com outputs $[\mathbf{a}_j]_i = \mathbf{a}_j(i)$, and each honest node $\mathcal{P}'_i$ in com' outputs $[\bar{\mathbf{a}}_j]_i = \bar{\mathbf{a}}_j(i)$. Moreover, each honest node $\mathcal{P}_i$ in com will deterministically extract $[\mathbf{u}]_i = \mathbf{u}(i)$ through Vandermonde matrix, and each honest node $\mathcal{P}'_i$ in com' will extract $[\bar{\mathbf{u}}]_i = \bar{\mathbf{u}}(i)$, where $\mathbf{u}(x)$ is a vector of t -degree polynomials and $\bar{\mathbf{u}}(x)$ is a vector of t' -degree polynomials, which satisfy $\mathbf{u}(0) = \bar{\mathbf{u}}(0)$.

Argument 3: honest nodes in com' output the same masked secrets and GoSSamer-ADPSS satisfies correctness.

Each honest node $\mathcal{P}_i$ in the old committee calculates $[s]_i + [u]_i$ and constructs the polynomials $F(x, i)$. By assumption, the secret share $[s]_i$ of node $\mathcal{P}_i$ satisfies $[s]_i = f(i)$, so the following equation holds, and it is straightforward to verify that $F(x, i)$ is an encoding of $f(i) + u(i)$.

$$[s]_i + [u]_i = f(i) + u(i)$$

Each honest node $\mathcal{P}_i$ in the old committee then distributes $F(u, i)$ to each node $\mathcal{P}_u$ within its committee. This ensures each honest node $\mathcal{P}_u$ outputs $F(u, y)$ via the robust polynomial interpolation algorithm, and multicasts $F(u, 0)$ to the new committee com' . All honest nodes in the new committee then output $F(x, 0)$ from the robust polynomial interpolation algorithm, and decode it into $f(0) + u(0)$, which satisfies:

$$f(0) + u(0) = s + u$$

The reshared secret share is then calculated as:

$$[\bar{s}]_i = s + u - [\bar{u}]_i = s + u - \bar{u}(i)$$

Thus, $[\bar{s}]_i$ corresponds to the evaluations $\bar{f}(i)$ of the t' -degree polynomials $\bar{f}(x)$, where the following equation holds for $\bar{f}(x)$.

$$\bar{f}(x) = s + u - \bar{u}(x), \bar{f}(0) = s$$

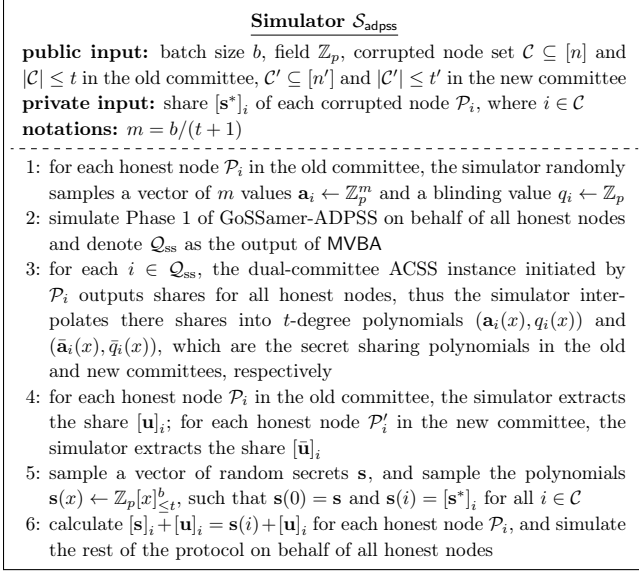


Figure 9: The simulator $\mathcal{S}_{\text{adpss}}$ of GoSSamer-ADPSS

This concludes the proof that GoSSamer-ADPSS satisfies the correctness property of ADPSS. $\square$

Lemma 10 (Secrecy). GoSSamer-ADPSS satisfies the secrecy property of ADPSS. Specifically, there exists a P.P.T simulator $\mathcal{S}_{\text{adpss}}$ which, given the shares of the corrupted nodes in the old committee, it can produce a simulated view in the ideal world with a vector of independently sampled secrets, such that the ideal and real worlds are indistinguishable from the view of the adversary $\mathcal{A}$.

Proof of Lemma 10. We employ a hybrid argument to prove that the view of the adversary $\mathcal{A}$ in the real execution with actual secrets is indistinguishable from its view in the ideal world. In the ideal world, the simulator $\mathcal{S}_{\text{adpss}}$ is given only the secret shares held by corrupted nodes and simulates using uniformly sampled secrets, as summarized in Figure 9.

Hybrid 0. This is a real-world execution of GoSSamer-ADPSS, where all honest nodes adhere to the protocol.

Hybrid 1. Same as Hybrid 0, except that the honest nodes are simulated and honestly follow the protocol. For each honest node $\mathcal{P}_i$ in the old committee com, let $\mathbf{a}_i$ denote a vector of independently sampled values, let q_i denote an independently sampled blinding value. The pair $(\mathbf{a}_i, q_i)$ constitutes the input to the dual-committee ACSS protocol for node $\mathcal{P}_i$. Hybrid 1 remains indistinguishable from Hybrid 0, as the distributions of $(\mathbf{a}_i, q_i)$ for all $i \in \mathcal{H}$ are identical in both hybrids.

Hybrid 2. Same as Hybrid 1, except that all honest nodes are simulated by the simulator $\mathcal{S}_{\text{adpss}}$ specified in Figure 9. The differences between Hybrid 2 and Hybrid 1 occur in Lines 5-6 of Figure 9, where original secrets $\mathbf{s}^*$ are replaced with the

randomly sampled secrets $\mathbf{s}$, and the masked share for each honest node $\mathcal{P}_i$ in the old committee is computed as $[\mathbf{s}]_i + [\mathbf{u}]_i$.

In the old committee, each node $\mathcal{P}_i$ interpolates the masked share $[\mathbf{s}]_i + [\mathbf{u}]_i$ into the t -degree polynomials $\mathbf{F}(x, i)$, and distributes each evaluation $\mathbf{F}(j, i)$ to each node $\mathcal{P}_j$ within the old committee. Moreover, each node $\mathcal{P}_j$ in the old committee multicasts the interpolated $\mathbf{F}(j, 0)$ to all nodes in the new committee. Let $\mathcal{H} = [n]/\mathcal{C}$ denote the set of honest nodes in the old committee, and $\mathcal{H}' = [n']/\mathcal{C}'$ in the new committee. The view of the adversary $\mathcal{A}$ in both hybrids differs for: (i) $\mathbf{F}(j, i)$ for all $j \in \mathcal{C}$ and $i \in \mathcal{H}$, and (ii) $\mathbf{F}(j, 0)$ for all $j \in \mathcal{H}$. These polynomial evaluations do not reveal any information about $\mathbf{s}$ beyond what is available from $\mathbf{s} + \mathbf{u}$. Furthermore, the secrecy property of dual-committee ACSS ensures that $[\mathbf{u}]_i$, for any $i \in \mathcal{C}$ or $i \in \mathcal{C}'$, discloses no information about $\mathbf{u}$. Consequently, Hybrid 2 is indistinguishable from Hybrid 1. The sequence of indistinguishable hybrids implies that the adversary cannot distinguish between the real and ideal executions. Hence, GoSSamer-ADPSS satisfies the secrecy property. $\square$

Theorem 5. GoSSamer-ADPSS is an ADPSS protocol that satisfies correctness, termination, and secrecy properties against the adversaries as defined in Section 2.4.

Proof of Theorem 5. The theorem follows directly from Lemma 8, Lemma 9, and Lemma 10. $\square$